Check for updates

ACM DIGITAL LIBRARY

Association for Computing Machinery

acm open

Latest updates: https://dl.acm.org/doi/10.1145/3744919

PDF Download
3744919.pdf
14 February 2026
Total Citations: 0
Total Downloads: 248

Published: 13 February 2026
Online AM: 23 June 2025
Accepted: 04 June 2025
Revised: 03 June 2025
Received: 14 June 2024

Citation in BibTeX format

RESEARCH-ARTICLE

POS Tagging on Code Identifiers: How Far Are We?

HANLIN TANG, Beijing Institute of Technology, Beijing, China

YANJIE JIANG, Tianjin University, Tianjin, China

YUXIA ZHANG, Beijing Institute of Technology, Beijing, China

NAN NIU, University of Cincinnati, Cincinnati, OH, United States

HUI LIU, Beijing Institute of Technology, Beijing, China

Open Access Support provided by:

Tianjin University

Beijing Institute of Technology

University of Cincinnati

ACM Transactions on Software Engineering and Methodology, Volume 35, Issue 3 (March 2026)
https://doi.org/10.1145/3744919
EISSN: 1557-7392

POS Tagging on Code Identifiers: How Far Are We?

HANLIN TANG, School of Computer Science and Technology, Beijing Institute of Technology, Beijing, China
YANJIE JIANG, College of Intelligence and Computing, Tianjin University, Tianjin, China
YUXIA ZHANG, School of Computer Science and Technology, Beijing Institute of Technology, Beijing, China
NAN NIU, EECS, University of Cincinnati, Cincinnati, Ohio, USA
HUI LIU, School of Computer Science and Technology, Beijing Institute of Technology, Beijing, China

Part-of-Speech (POS) tags are natural attributes of words in natural languages, and they are fundamental for natural language analysis. Many automated approaches have been proposed to tag natural language texts. Identifiers in source code have POS tags as well, which are useful for various source code analysis tasks, like code search, code comment generation, and code completion. Currently, state-of-the-art POS taggers originally designed for natural languages are often employed to tag source code identifiers. However, identifiers in source code are significantly different from natural languages. Consequently, POS taggers designed for natural languages could be less accurate in source code identifiers. Recently, several identifier-specific taggers have been proposed within the field of software engineering, but their adoption in practical software engineering tasks remains limited. This raises the question of why these taggers have not been more widely utilized in such tasks.

In this article, we investigate the performance of natural language POS taggers on source code identifiers, specifically method names, parameter names, and class names. To do so, we manually annotated identifiers from open source projects in Java, C, and Python, creating a large dataset IDData for evaluation. We then evaluated six widely used natural language POS taggers: NLTK, CoreNLP, OpenNLP, spaCy, Flair, and Stanza, alongside three identifier-specific taggers: SWUM, POSSE, and Ensemble Tagger. Our evaluation reveals that while natural language-oriented POS taggers outperform identifier-specific taggers, their performance on identifiers is still significantly lower compared to their performance on natural language sentences. To understand the underlying reasons for this, we conducted an in-depth analysis, examining factors such as identifier length, POS distribution, syntactic structures, and special tags, which differentiate identifiers from natural language sentences. To further improve POS tagging performance on identifiers, we created a large-scale method name dataset MNTrain with manually labeled tags and retrained the natural language taggers on this new dataset. The results show substantial improvements in method name POS tagging performance, with taggers achieving performance comparable to their results on natural language sentences. Finally, we discuss the significance and practical implications of our findings, offering insights for future research.

This work was partially supported by the National Natural Science Foundation of China (Grant Nos. 62232003 and 62172037) and the China National Postdoctoral Program for Innovative Talents (Grant No. BX20240008).
Authors' Contact Information: Hanlin Tang, School of Computer Science and Technology, Beijing Institute of Technology, Beijing, China; e-mail: hltang@bit.edu.cn; Yanjie Jiang (corresponding author), College of Intelligence and Computing, Tianjin University, Tianjin, China; e-mail: jiangyanjiese@gmail.com; Yuxia Zhang, School of Computer Science and Technology, Beijing Institute of Technology, Beijing, China; e-mail: yuxiazh@bit.edu.cn; Nan Niu, EECS, University of Cincinnati, Cincinnati, Ohio, USA; e-mail: nan.niu@uc.edu; Hui Liu, School of Computer Science and Technology, Beijing Institute of Technology, Beijing, China; e-mail: liuhui08@bit.edu.cn.
Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM 1557-7392/2026/2-ART84
https://doi.org/10.1145/3744919

CCS Concepts: • **General and reference** → **Empirical studies**; • **Software and its engineering** → **Empirical software validation**;

Additional Key Words and Phrases: Part-of-speech Tagging, Code Identifier, Evaluation of Tools, Natural Language Processing

ACM Reference format:
Hanlin Tang, Yanjie Jiang, Yuxia Zhang, Nan Niu, and Hui Liu. 2026. POS Tagging on Code Identifiers: How Far Are We?. *ACM Trans. Softw. Eng. Methodol.* 35, 3, Article 84 (February 2026), 33 pages.
https://doi.org/10.1145/3744919

1 Introduction

Part-of-Speech (POS) tags refer to the grammatical categories or syntactic roles of words within a sentence, such as nouns, verbs, adjectives, and adverbs [42]. POS tags are critical for understanding the syntactic structure of sentences and extracting semantics from normal text, serving as a fundamental component in various **Natural Language Processing (NLP)** tasks, including text classification [53], named entity recognition [33], and machine translation [49]. The POS tagging task involves labeling words with their respective parts of speech. Existing studies have proposed many automated POS tagging approaches based on machine learning techniques. Typical taggers include NLTK [11], Stanford CoreNLP [39], Apache OpenNLP [6], spaCy [18], Flair [2], and Stanza [50]. POS tags are also valuable for code analysis [46]. Source code, especially identifiers like method names, are often regarded as special natural languages. Consequently, tokens (words) within identifiers have their POS tags as well. Such tags have been leveraged for various software engineering tasks, like code search [36], comment generation [65], and issue report analysis [63]. To facilitate the task of code analysis, in this article, we study POS tagging on identifiers.

There are two ways to do POS tagging on identifiers. The first is directly using the off-the-shelf natural language taggers, and the other is to design identifier-specific taggers. Many researchers [32] directly reuse the natural language-oriented POS taggers like Stanford CoreNLP [39]. Such POS taggers have been well recognized and widely used because of their high accuracy in tagging natural languages and public availability. In recent years, deep learning-based taggers like spaCy and Stanza have been released. However, such off-the-shelf taggers were originally designed for natural languages and trained with natural language texts, which may affect their performance on identifiers because code identifiers are essentially different from natural language sentences in terms of their lengths and structures. To improve the tagging performance on identifiers, a few POS taggers have been proposed specifically for source code identifiers [21, 23, 47]. For example, Newman et al. [47] proposed Ensemble Tagger, which integrates three POS taggers by the Decision Tree [45] and Random Forest [9]. Additionally, Olney et al. [48] evaluated several POS taggers on 247 Java method names. Their results suggested that identifier-specific taggers outperform natural language taggers like Stanford CoreNLP on the identifier POS tagging task.

However, it is in doubt which type of taggers is more accurate in identifier POS tagging. The evaluation by Olney et al. [48] seems biased because of the following four reasons: First, their evaluation only involves one identifier type and one programming language. Second, the number of identifiers that they tested is limited. Third, some state-of-the-art taggers that were proposed recently have not been evaluated in their study. Fourth, the natural language taggers may achieve better performance if they are trained on large-scale identifier data. Thus, we still lack a large-scale empirical study to investigate whether such identifier-specific POS taggers are more accurate than natural language taggers. It is also necessary to explore whether natural language POS taggers can be accurate on identifiers, and if not, what are the reasons and how to improve them.

To this end, in this article, we conduct a large-scale empirical study to evaluate POS taggers with code identifiers. We manually construct a large-scale testing dataset composed of 1,603 method names, 663 parameter names, and 2,596 class names, where each token is explicitly labeled with a POS tag. With the resulting testing dataset, we investigate how well the six natural language oriented state-of-the-art POS taggers, i.e., NLTK, CoreNLP, OpenNLP, spaCy, Flair, and Stanza, work on code identifiers compared to their performance on natural language sentences. We also compare them against the state-of-the-art identifier-specific POS taggers, i.e., **Software Word Usage Model (SWUM)** [23], POSSE [21], and Ensemble Tagger [47], to investigate whether it is deserving to design identifier-specific POS taggers. Our evaluation results suggest that the identifier-specific POS taggers are less accurate on code identifiers than natural language-oriented state-of-the-art POS taggers. We also observe that the performance of the natural language-oriented state-of-the-art POS taggers is reduced substantially on code identifiers compared to their performance on natural language sentences. To reveal the potential reasons for the reduction, we compare the code identifiers against natural language sentences and reveal how different they are concerning lengths, POS distribution, syntactic structures, and special tags. Our findings suggest that compared to natural language sentences, code identifiers have unique characteristics, e.g., much shorter lengths, incomplete structures, and special domain-related tokens, which indicates the necessity of training natural language POS taggers using identifier POS datasets. Consequently, we manually construct a large-scale high-quality dataset that contains method names and their POS tags and retrain the natural language POS taggers. After the retraining, the performance of the natural language POS taggers on method names is comparable to their corresponding performance on natural language sentences.

The main contributions of this article are listed as follows:

- We conduct the first large-scale empirical study of POS tagging on code identifiers. The empirical study results in several insightful findings that may guide future research and practice in this field. The replication package is publicly available on GitHub [55] for third-party replication and further evaluation of POS taggers.
- We construct two datasets for code identifier POS tagging: IDData and MNTrain, which adopt the same tagset with generic natural language corpus, i.e., the Penn Treebank tagset [52]. IDData is introduced as a new benchmark for evaluating identifier POS tagging. It consists of a test dataset containing 1,603 method names, 663 parameter names, and 2,596 class names, collected from 9 open source projects. MNTrain contains 6,165 method names from 10 open source projects, which are used to retrain natural language taggers. Each token in the two datasets has been manually labeled with a POS tag. IDData serves as a testing dataset for evaluating POS taggers, while MNTrain provides an effective means to enhance existing POS taggers for method name POS tagging. The two datasets are publicly available on GitHub [55], which can assist researchers in further improving POS taggers.

The rest of the article is organized as follows: Section 2 introduces the related work. Section 3 presents the design of the empirical study, whereas Section 4 presents the results and analysis. Section 5 presents how we improve the natural language-oriented POS taggers with the MNTrain dataset. Section 6 discusses the threats to validity and limitations, and Section 7 makes the conclusions.

2 Related Works

2.1 POS Tagging in NLP

There are three types of approaches for POS tagging: rule-based, statistical learning-based, and deep learning-based approaches. Brill [12, 13] proposed a rule-based approach, where researchers

manually designed tagging rules based on context and syntactic structure. Brill [14] also found that such rules can be mined from natural language corpus using transformation-based approaches.

Statistical learning-based approaches have been widely employed for POS tagging. Kupiec [31] and Cutting [16] tagged words by learning the probability distribution of sequences of words and POS tags using **Hidden Markov Models (HMMs)**. Banko and Moore [7] extended the HMM-based approach with sequential unsupervised training of tag and lexical information. Ratnaparkhi [51] applied the **Maximum Entropy (ME)** model for POS tagging with a flexible feature set from contexts. Toutanova and Manning [59] enriched the ME-based POS tagger with extra capitalization and disambiguation of tokens. Toutanova et al. [58] built a more fine-grained tag dependency and lexical feature usage to present a cyclic dependency network-based POS tagger. Other learning models like support vector machines [43], decision trees [41], and Bayesian models [20] were also employed to learn the patterns in language data and automatically assign POS tags based on those patterns [42].

In recent years, deep learning techniques have been employed for POS tagging as well. Ling et al. [34] leveraged bi-LSTMs to capture the lexical and regular morphological knowledge of character sequences. Liu et al. [35] utilized both word-level and character-level pre-trained embedding from language models to empower POS taggers. They also integrated a **Conditional Random Field (CRF)** to promote the LSTM-based tagger. Jung et al. [28] proposed an ensemble candidate generation framework containing POS tagging using a pre-trained encoder.

2.2 POS Tagging in Software Engineering

2.2.1 Technologies. Some POS taggers have been designed for code identifiers. These taggers are usually based on existing natural language-oriented POS taggers because of their notable performance in the NLP field. SWUM [23] tags tokens by the morphological parser PC-KIMMO [5] with further disambiguation rules. Gupta et al. [21] proposed the POSSE tagger, which assigns tags based on WordNet and morphological rules summarized from source code. This tagger utilizes a new POS tagset with 12 tags for code identifiers based on their collected naming conventions of identifiers. Alsuhaibani et al. [4] proposed a set of heuristics to tag source code identifiers. Machine learning models have also been leveraged in this task. Newman et al. [47] proposed an ensemble tagging approach for code identifiers. This tagger utilizes two machine learning models for tagging words, where the input is the output of several existing taggers like SWUM, POSSE, and Stanford CoreNLP. The results show that the state-of-the-art performance in tagging the POS of code identifiers. They also released a larger dataset for evaluation.

2.2.2 Applications. With the strengthening of AI technology, POS tags or POS taggers become more valued for the development of automated software engineering like the tasks of API documentation augmentation [60], defect localization [57], and comment generation [65]. Binkley et al. [10] used POS information to improve the identifier informativeness. They used Stanford POS Tagger to get the POS information of field names. Tian and Lo [56] conducted an empirical study on POS tagging on bug reports. Zhou et al. [64] leveraged POS information to augment bug localization, where they utilized Stanford POS Tagger to tag bug reports.

POS tagging is also widely used in requirement engineering. Bhowmik et al. [8] proposed a topic modeling framework that extracts familiar ideas from the requirements and stakeholders' comments. They identified the common POS for topic words in existing requirements and comments. Abad et al. [1] proposed an approach to promote automated classification of requirements, where they utilized Stanford Parser to get POS tags.

2.3 Evaluation of POS Taggers

Several studies were conducted to evaluate POS taggers. Jin et al. [27] conducted metamorphic testing on three POS taggers, i.e., spaCy, NLTK, and Flair. Specifically, they design four metamorphic

relations regarding both intra/inter-sentence properties. Their results demonstrated that this testing approach is an effective way to reveal the potential issues of POS taggers. Olney et al. [48] evaluated nine POS taggers on 247 Java method names, and their results suggested that the identifier-oriented POS tagger POSSE [21] was more accurate than natural language-oriented POS taggers. Our work differs from their work in the following aspects. First, our dataset is much larger than theirs. Our testing dataset is composed of 1,603 method names, 663 parameter names, and 2,596 class names from 9 open source applications, making it over 19 times larger than their dataset. They also lack evaluations on other program languages and other identifier types. Additionally, we build a bigger dataset for retraining POS taggers, which contains 10,000 method names with POS tags. Second, we investigate more research questions that yet have not been investigated by their work. For example, we investigate the difference between identifiers and natural language sentences, and to what extent the retraining of natural language-oriented POS taggers with method name corpus can improve their performance on method names. Third, we analyze state-of-the-art POS taggers (e.g., Flair [2], Stanza [50], and Ensemble Tagger [47]), which are not available when their evaluation was conducted.

In our empirical study, we adapt the union of the used POS taggers within the above studies as research objects, i.e., five state-of-the-art POS taggers: NLTK, CoreNLP, OpenNLP, spaCy, and Flair, and add a new state-of-the-art tagger Stanza. Instead of evaluating POS taggers solely on open source Java method names, as done by Olney et al. [48], we conduct the evaluation on three types of identifiers: method names, parameters, and class names.

3 Setup

In this section, we specify how we conduct a large-scale empirical study to evaluate the performance of natural language-oriented natural language POS taggers when they are applied to source code identifiers in comparison to identifier-specific taggers.

3.1 Research Questions

The empirical study investigates the research questions below:

- *RQ1*: How do state-of-the-art natural language-oriented POS taggers perform on identifiers?
- *RQ2*: How do the identifier-specific POS taggers perform comparing with the state-of-the-art natural language-oriented POS tagger?
- *RQ3*: Which POS tags are more challenging to label?
- *RQ4*: Why are natural language-oriented POS taggers less accurate on identifiers than on natural language sentences?

RQ1 concerns whether natural language-oriented POS taggers can tag identifiers accurately. Notably, source code identifiers including method names, parameters, and class names essentially differ from natural language sentences. Consequently, POS taggers trained with natural language corpus may not work accurately on identifiers. Answering this question can validate the necessity of designing identifier-specific POS taggers.

We select six widely used natural language-oriented POS tagger, i.e., NLTK, Stanford CoreNLP, Apache OpenNLP, spaCy, Flair, and Stanza as state-of-the-art POS taggers. To our knowledge, Flair and Stanza have not ever evaluated for identifier POS tagging. We briefly introduce the selected taggers as follows:

- NLTK (version: 3.7) [11]: NLTK is a popular Python NLP package developed by the University of Pennsylvania. It is the most famous toolkit in POS tagging because of its abundant models. We use its default tagger (Perception model) in our experiments.

- Stanford CoreNLP (version: 4.5.4) [39]: Stanford CoreNLP is a powerful NLP toolkit developed by the Stanford NLP Group. It contains a mature Maxent POS tagger.
- Apache OpenNLP (version: 2.3.2) [6]: Apache OpenNLP is an NLP tasks-aimed machine learning toolkit. It supports many tasks including POS tagging. Its default POS tagger with PTB tagset is a Maxent model.
- spaCy (version: 3.3.1) [18]: spaCy is an open source NLP library that provides a high-performance POS tagger. The model used in its official documents "en_core_web_sm" is selected.
- Flair (version: 0.12.2) [2]: Flair is a light framework for NLP. We select its "pos" model for evaluation.
- Stanza (version: 1.8.2) [50]: Stanza is a new Python NLP package with pipeline designing. It provides highly accurate neural network components with GPU support.

Notably, these taggers are selected not only for their state-of-the-art performance but also for their public availability and ongoing maintenance.

RQ2 studies the performance of current identifier-specific POS taggers. Answering RQ2 will reveal which type of POS taggers (identifier-specific taggers vs. natural language taggers) has better performance.

RQ3 concerns the accuracy of natural language POS taggers on different categories of POS tags and the patterns of POS tags in identifiers. Likely, some categories of POS tags and patterns are easier to tag than others. By digging into different categories of POS tags, we may reveal the direction for future improvement.

The first three research questions concern the performance of the taggers, and RQ4 concerns the reason for the performance reduction. The revealed reasons could be employed to guide the design of identifier-specific POS taggers.

3.2 Datasets

The empirical study applies the evaluated taggers to natural language sentences and identifiers. Notably, all data items should be labeled with POS tags. To collect natural language sentences (noted as NLdata), we randomly sample 3,000 English sentences from the **Georgetown University Multilayer (GUM)** corpus [62]. GUM corpus comes from the open source cross-linguistically project **Universal Dependencies (UD)** [17], and it is one of the most frequently used corpora in POS tagging [17]. Notably, we extract the tags of sampled sentences from GUM, and these tags serve as the gold standard in the empirical study.

To the best of our knowledge, there is a lack of a large-scale high-quality dataset with identifiers and their POS tags for identifier POS tagging. To this end, we create such a dataset from scratch. The creation is explained as follows:

- *Data Collection.* We select nine popular projects, i.e., Hadoop, OpenJDK, Maven, Obs-studio, Mpv, Redis, Django, Keras, and Pandas. The projects are written in three different programming languages, i.e., Java, C, and Python. We first parse the source code within the selected projects and extract the identifiers in the source code. Notably, to ensure the generalization of the collected data, we collect three different types of identifiers, i.e., method names, parameter names, and class names. For identifiers in different programming languages, the extraction processes need different tools. We leverage JavaParser [25], srcML [15], and the ast [19] library to extract identifiers from Java, C, and Python projects, respectively. Given the significant time and effort required for manual annotation, we sample 1,000 identifiers from each project instead of annotating all of them. The sampled data have a 95% confidence interval of [0.99,1].

— *Tokenization.* The sampled identifiers are tokenized with the spiral [24] library, which is an identifier-specific tokenizer.
— *Tagset Selection.* We annotate identifiers using the Penn Treebank POS tagset [52] and other tagsets used for identifiers, e.g., the custom tagset introduced by Newman et al. [47] (referred to as Newman's tagset). The reasons are listed as follows:
  (1) *Broad Compatibility with Natural Language-Oriented Taggers.* The Penn Treebank tagset is natively supported by most widely used natural language-oriented POS taggers, and crucially, it is the only tagset supported by all the taggers evaluated in our study. This broad compatibility allows for consistent comparison across different taggers, facilitating an equitable evaluation of their performance.
  (2) *Mapping between Tagsets.* Newman's tagset is derived from a manually curated mapping based on the Penn Treebank tagset [46]. While the Penn Treebank tags can be transformed into Newman's tagset through the mapping, the reverse transformation is not possible. This limitation makes Newman's tagset less flexible for comparative analysis. By using the Penn Treebank tagset, we ensure that the tagsets used by all taggers are comparable, enabling us to evaluate the performance of both natural language-oriented and identifier-specific taggers.
— *Annotation.* The sampled identifiers are manually annotated by three annotators with experience in programming and sequence labeling tasks within NLP. Specifically, each annotator has a minimum of 5 years of programming experience, ensuring that the annotators are well equipped to handle the intricacies of identifier annotation. All annotators have prior exposure to sequence labeling tasks and are familiar with the structure and syntax of programming identifiers. Before beginning the annotation process, we organized learning sessions on POS categories, following the Penn Treebank POS guidelines [52]. Each annotator independently tags the identifiers, and they are given access to the source of the identifiers to cross-check contextual information. To maintain linguistic consistency, every token found in the Concise Oxford English Dictionary [54] was assigned a single POS tag based on the annotators' collective judgment. In cases where discrepancies occurred between individual annotations, a resolution process was initiated, in which the entire annotation group reviewed and discussed the conflicts. Final decisions are made collaboratively to ensure the integrity of the dataset.
— *Special Annotations.* Several special cases are addressed during the annotation process to ensure the precise tagging of identifiers:
  (1) *Proper Nouns.* Identifiers representing proper nouns are tagged either as "NNP" or "NNPS," depending on whether they are singular or plural.
  (2) *Expandable Character Sequences.* If a character sequence (e.g., abbreviations or shorthand commonly used in programming) could be expanded into a full word or phrase, it is tagged using the same POS label as the expanded form.
  (3) *Project-Related Abbreviations.* Other character sequences (project-specific abbreviations) that could not be expanded or recognized by common dictionaries or general knowledge are tagged as "FW" (foreign words).
— *Duplicate Removal Strategy.* To ensure a more accurate assessment of the tagger performance, we implemented a duplicate removal strategy. Specifically, we removed any duplicated items within the dataset. In cases where multiple identifiers with the same names, we retained only the first occurrence of each identifier and its associated annotation. By removing such duplication, the evaluation focuses on unique instances, ensuring that the taggers are being tested on a more representative and diverse set of inputs.

An overview of the resulting datasets is presented in Table 1. The testing dataset is composed of two parts: the identifier dataset (IDData) and the natural language dataset (NLData). As previously

Table 1. Dataset Statistics

Dataset	Project	#Identifiers	#Tokens	#Unique Tags
MData	Hadoop	709	2,130	23
	OpenJDK	373	965	17
	Maven	521	1,455	15
	In Total	1,603	4,550	23
PData	Redis	231	394	17
	Mpv	237	349	13
	Obs-studio	195	280	11
	In Total	663	1,023	17
CData	Django	856	2,106	22
	Keras	869	2,224	21
	Pandas	871	2,700	21
	In Total	2,596	7,030	22
IDData	In Total	4,862	12,603	23
NLData	UD-GUM	3,000	49,078	46

mentioned, the NLData represents a sampling subset from the GUM corpus, and the IDData is gathered from open sourced projects. We select three types of identifiers: method names, parameter names, and class names. All identifiers are sampled from the most popular open sourced projects from GitHub. The method name testing dataset (MData) consists of 1,603 method names, reduced from an original pool of 3,000 entries after removing duplicates. Similarly, the parameter name testing dataset (PData) contains 663 annotated parameter names and the class name testing dataset (CData) includes 2,596 annotated class names. We aim to form a large-scale testing dataset for the identifier POS tagging task.

The table highlights that natural language sentences are substantially longer than identifiers: the former on average (16.36 tokens) is more than six times longer than the latter (2.59 tokens). The length of parameter names is even shorter on average (1.54 tokens). Furthermore, it is observed that the count of distinctive tags in identifiers (23) is notably fewer compared to those in natural language sentences (46).

3.3 Process

To investigate how the natural language-oriented POS taggers work on identifiers, we apply all of the selected taggers independently of the three identifier testing datasets MData, PData, and CData, and validate the quality of the tagging by comparing the predicted tags against the corresponding tags in the dataset. If and only if the predicted tags are identical to those in the dataset, the prediction (tagging) is correct. Notably, to compare the performance of the taggers on three types of identifiers against the performance on natural languages, we also apply the selected POS taggers independently of the natural language dataset (NLData).

Note that all taggers are set to use their default English tagging model. The evaluation is conducted on an Ubuntu 18.04 server with Intel(R) Xeon(R) CPU E5-2620 v4 @ 2.10GHz, 128G RAM, and 4x NVIDIA RTX 2080Ti.

We use the following performance metrics to evaluate the performance of POS tagging:

Table 2. Performance of Taggers (Token Accuracy)

Taggers	NLData	MData	PData	CData	IDData	$\Delta(NL - ID)$
NLTK	88.68%	67.69%	67.16%	64.25%	65.73%	22.95%
CoreNLP	95.02%	76.70%	67.35%	70.17%	72.30%	22.72%
OpenNLP	91.38%	74.29%	65.88%	67.82%	70.00%	21.38%
spaCy	95.52%	73.34%	53.57%	62.29%	65.57%	29.95%
Flair	95.52%	78.13%	63.25%	70.55%	72.70%	22.82%
Stanza	**96.70%**	**83.76%**	**68.52%**	**72.84%**	**76.43%**	20.27%

The bold and underlined font indicates the best-performing natural language-oriented tagger for each dataset.

Table 3. Performance of Taggers (Exact Match)

Taggers	NLData	MData	PData	CData	IDData	$\Delta(NL - ID)$
NLTK	29.60%	28.57%	55.35%	32.86%	34.51%	−4.91%
CoreNLP	53.20%	49.10%	56.26%	41.60%	46.07%	7.13%
OpenNLP	37.43%	43.79%	54.75%	36.83%	41.57%	−4.13%
SpaCy	56.47%	44.73%	41.03%	33.90%	38.44%	18.03%
Flair	58.37%	51.65%	52.49%	42.18%	46.71%	11.66%
Stanza	**74.07%**	**64.19%**	**56.71%**	**45.18%**	**53.02%**	21.04%

The bold and underlined font indicates the best-performing natural language-oriented tagger for each dataset.

— Token Accuracy (Acc_{Token}) is the most common metric in POS tagging [29]. It measures the percentage of correctly tagged tokens.

$$Acc_{Token} = \frac{Number\ of\ Correctly\ Tagged\ Tokens}{Total\ Number\ of\ Tokens}. \tag{1}$$

— Exact Match (EM) is an accuracy metric at the identifier/sentence level. It measures the percentage of correctly tagged identifiers/sentences.

$$EM = \frac{Number\ of\ Correctly\ Tagged\ Identifiers\ or\ Sentences}{Total\ Number\ of\ Identifiers\ or\ Sentences}. \tag{2}$$

4 Results and Analysis

4.1 RQ1: Reduced Performance

We apply the six state-of-the-art natural language-oriented POS taggers to both identifiers and natural language corpus as specified in Section 3. The results are presented in Tables 2 and 3. From the tables, we make the following observations:

— First, the performance of the natural language-oriented POS taggers at the token level reduces substantially when applied to identifiers. As suggested by Table 2, the token-level accuracy Acc_{Token} of the evaluated taggers was reduced by at least 20.27%. For example, the Acc_{Token} of spaCy was reduced from 95.52% (on natural language sentences) to 65.57% (on identifiers), with a relative reduction of $31.35\% \approx (95.52\% - 65.57\%)/95.52\%$.

— The reduction in EM is not as substantial as that in Acc_{Token}. We notice that some of the most advanced taggers, e.g., Stanza, Flair, and spaCy, result in substantially reduced EM scores

when they are applied to source code identifiers. However, we also observe that some taggers (e.g., NLTK and OpenNLP) even perform better on identifiers than natural language sentences. One possible reason for the surprising improvement is that identifiers on average are much shorter than natural language sentences, and thus, they have much greater chances to reach an exact matching (i.e., all tokens within the given identifier are tagged correctly) even if the taggers perform equally well on identifier tokens and natural language tokens.

- The best taggers on source code identifiers in terms of Acc_{Token} and EM are those that have the best performance on natural language sentences. Stanza demonstrates leadership in both natural language sentences and identifiers. It may suggest that the POS of source code is somewhat similar to that of natural language sentences, although they are not identical.
- The Token Accuracy of the taggers on parameter names is significantly lower compared to their performance on other identifier types, such as method names and class names. This discrepancy suggests that tagging parameter tokens presents greater challenges. We think that this difficulty arises from the fact that parameter tokens tend to lack sufficient contextual information, primarily because they have the shortest average length compared to other identifier types. Without the richness of neighboring token information, it becomes harder for taggers to accurately predict the correct POS for parameter tokens.
- Additionally, we observe that Stanza now achieves the best performance across all identifier categories, including parameter names, slightly outperforming CoreNLP by less than one percentage point in terms of Exact Match. This result suggests that while deep learning-based taggers such as Stanza can effectively leverage contextual information even for shorter sequences, statistical learning-based models like CoreNLP still demonstrate competitive performance. The relatively strong performance of CoreNLP on parameter names indicates that traditional rule-based or statistical methods remain effective in scenarios where contextual cues are limited, such as in short identifier sequences.
- Significant improvement in POS taggers for identifier tagging is required. Identifiers are much shorter than natural language sentences, so the performance on identifiers should have been better than that on natural language sentences. However, the state-of-the-art POS tagger Stanza can only tag 53% of the identifiers in terms of EM, which is significantly lower than its EM on natural language sentences.

Finding 1: The performance of the natural language-oriented POS taggers is substantially reduced when applied to source code identifiers, and thus, it deserves substantial improvement to make the taggers useful on identifiers.

4.2 RQ2: Identifier-Specific Taggers

4.2.1 Performance on IDData. As suggested in Section 4.1, the performance of the natural language-oriented POS taggers has been substantially reduced when applied to source code identifiers. It may suggest that the tagging of code identifiers could be substantially different from the tagging of natural languages. Consequently, it is likely that the state-of-the-art POS taggers specially designed for source code identifiers (e.g., Ensemble Tagger [47], SWUM [23] and POSSE [21]) may significantly outperform the natural language-oriented POS taggers originally designed for natural languages. To validate this hypothesis, we compare those identifier-specific taggers with natural language-oriented taggers on the same datasets that we use to answer RQ1.

During the evaluation phase, we map the Penn Treebank tagset [52] into smaller Newman's tagset to ensure a fair comparison. The Penn Treebank tagset can be mapped into that reduced

Table 4. Performance of Identifier-Specific Taggers (Token Accuracy)

Taggers	MData	MData-unk	PData	PData-unk	CData	CData-unk	IDData	IDData-unk
SWUM	62.18%	62.26%	55.43%	56.10%	39.43%	39.49%	48.94%	49.06%
POSSE	**77.96%**	**77.99%**	**76.74%**	**77.60%**	**70.34%**	**70.46%**	**73.61%**	**73.76%**
Ensemble Tagger	70.40%	70.49%	61.49%	62.24%	49.05%	49.13%	57.76%	57.91%
NLTK[a]	71.56%	71.54%	84.36%	84.64%	74.07%	74.09%	74.00%	74.02%
CoreNLP[a]	82.09%	82.09%	**85.14%**	**85.33%**	80.31%	80.33%	81.35%	81.37%
OpenNLP[a]	79.12%	79.18%	81.43%	81.76%	77.44%	77.45%	78.37%	78.42%
spaCy[a]	80.26%	80.24%	76.15%	76.31%	76.74%	76.74%	77.97%	77.97%
Flair[a]	82.90%	82.88%	79.18%	79.29%	78.73%	78.75%	80.27%	80.29%
Stanza[a]	**88.22%**	**88.23%**	84.07%	84.34%	**82.35%**	**82.37%**	**84.61%**	**84.65%**

[a]Indicates that the tags output by the natural language-oriented taggers has been mapped to the smaller tagset used by identifier-specific taggers.
The bold font indicates the best-performing identifier-specific tagger for each dataset.
The bold and underlined font indicates the best-performing natural language-oriented tagger for each dataset.

tagset, and the process is unidirectional (i.e., the conversion cannot be reversed). We implement an automatic mapping process that adheres to the same methodology proposed in the original work on the Ensemble Tagger [46, 47]. This mapping approach was utilized in their evaluation to compare two types of taggers. Without this mapping, it would be challenging to directly compare natural language-oriented POS taggers with identifier-specific taggers, as they operate using distinct tagsets. Some tags are not included in Newman's tagset, and these tokens are mapped into "UNK" (unknown) during their evaluation. In contrast, in our empirical study, we employ a standard and much larger tagset, i.e., Penn Treebank POS tagset [52]. To mitigate the impact of tokens being mapped to "UNK," we also conducted an evaluation excluding items containing this tag. Specifically, we filtered out 17 identifiers that have tokens mapped to "UNK" across IDData: 3 from MData, 7 from PData, and 7 from CData. By removing these identifiers, we aim to ensure that the evaluation results more accurately reflect the taggers' ability to handle known tokens, minimizing potential bias introduced by undefined mappings.

From Tables 4 and 5, we observe that Stanza, a state-of-the-art natural language POS tagger, significantly outperforms the identifier-specific taggers across all types of identifiers. Compared to the best identifier-specific tagger (i.e., POSSE), it improves the token-level Acc_{Token} from 73.76% to 84.65% and improves the identifier-level EM from 54.63% to 66.75%. One possible reason for the improvement is that natural language POS taggers are trained with much larger datasets, whereas identifier-specific taggers have limited training corpus.

An interesting observation from the evaluation is that the three statistical learning-based taggers (NLTK, CoreNLP, and OpenNLP) outperform deep learning-based taggers on the identifier datasets. Specifically, CoreNLP achieves the highest performance in Token Accuracy, while NLTK demonstrates the best results in terms of Exact Match. This performance difference may suggest that statistical models are better suited for shorter identifiers, where local token context is limited. In contrast, deep learning-based taggers, which typically excel in tasks with richer context and longer sequences, appear to rely more heavily on neighboring token information. The relatively short length of identifiers may thus limit the effectiveness of deep learning models, as they are unable to fully leverage their strengths in capturing long-range dependencies and contextual relationships.

Notably, our evaluation results are not consistent with those Olney et al. [48] reported. In their evaluation, the identifier-specific tagger (i.e., POSSE) was more accurate than natural language taggers trained with natural languages. One possible reason is that the two evaluations are conducted on different datasets. Notably, our dataset is much larger (4,862 vs. 247) than that used by Olney

Table 5. Performance of Identifier-Specific Taggers (Exact Match)

Taggers	MData	MData-unk	PData	PData-unk	CData	CData-unk	IDData	IDData-unk
SWUM	33.69%	33.75%	46.46%	46.95%	17.68%	17.73%	26.88%	26.98%
POSSE	**56.33%**	**56.44%**	**70.59%**	**71.34%**	**49.15%**	**49.29%**	**54.44%**	**54.63%**
Ensemble Tagger	43.04%	43.13%	49.47%	50.00%	20.18%	20.24%	31.72%	31.83%
NLTK*	32.25%	32.19%	**79.49%**	**79.88%**	46.80%	46.81%	46.46%	46.46%
CoreNLP*	57.45%	57.44%	79.19%	79.57%	56.63%	56.62%	59.98%	60.00%
OpenNLP*	51.78%	51.88%	74.21%	74.70%	51.69%	51.68%	54.79%	54.86%
spaCy*	53.21%	53.13%	65.76%	66.16%	51.16%	51.14%	53.83%	53.83%
Flair*	59.70%	59.63%	70.89%	71.19%	53.66%	53.65%	58.00%	58.00%
Stanza*	**72.86%**	**72.88%**	77.38%	77.74%	**60.17%**	**60.18%**	**66.70%**	**66.75%**

The bold font indicates the best-performing identifier-specific tagger for each dataset.
The bold and underlined font indicates the best-performing natural language-oriented tagger for each dataset.

et al. [48]. We also notice that compared to their dataset, our dataset has greater diversity: It is composed of multiple categories of identifiers (i.e., method names, class names, and parameter names), whereas their dataset is composed of method names only. Our dataset is extracted from Java, C, and Python programs, whereas their dataset is extracted from Java programs only. Another possible reason for the difference is that the natural language POS taggers have achieved huge improvements recently, which may have changed the conclusions.

4.2.2 Performance on Newman's Dataset. To ensure a fair evaluation, we applied the taggers to Newman's dataset [46, 47]. Newman's dataset was originally developed for the Ensemble Tagger and comprises a total of 1,335 identifiers for training and 384 identifiers for testing. The dataset is annotated using a narrower, customized tagset specific to identifier tagging, which differentiates it from broader tagsets commonly used in NLP. To avoid any bias introduced by mapping different tagsets, we trained the selected natural language-oriented POS taggers directly on Newman's dataset, ensuring that they could natively produce output with the same tagset. It is important to note that OpenNLP and Stanza were excluded from this process, as their architectures do not support the incorporation of custom tagsets.

We conducted a comprehensive evaluation by running both identifier-specific taggers and natural language-oriented POS taggers on the testing portion of Newman's dataset. The performance results, summarized in Table 6, reveal that the natural language-oriented POS taggers significantly outperform identifier-specific taggers. Among the identifier-specific taggers, the Ensemble Tagger achieves the best performance, but it still falls behind the natural language-oriented taggers by a considerable margin. Notably, Flair achieves the highest Token Accuracy (97.85%) and Exact Match (95.31%) on Newman's dataset, followed closely by spaCy. Interestingly, the performance of the natural language taggers on Newman's dataset is even higher than their performance on natural language sentences, as shown in Table 4, with results approaching near-perfect scores.

Finding 2: POS taggers specially designed for source identifiers are less accurate than the natural language-oriented POS taggers originally designed for natural languages.

4.3 RQ3: Mislabeled Items

To answer RQ3, we compute the performance of the POS taggers on different POS tags and patterns. Table 7 presents how their performance varies among different POS tags. For space limitation,

Table 6. Taggers' Performance on Newman's Dataset

Taggers	Token Accuracy	Exact Match
SWUM[a]	77.00%	58.00%
POSSE[a]	47.00%	20.00%
Ensemble Tagger[a]	**86.00%**	**75.00%**
NLTK	80.76%	63.02%
CoreNLP	92.53%	85.16%
spaCy	96.52%	92.71%
Flair	**97.85%**	**95.31%**

[a]Results from [47].
The bold font indicates the best-performing identifier-specific tagger for each dataset.
The bold and underlined font indicates the best-performing natural language-oriented tagger for each dataset.

Table 7. Top 5 Most Frequently Mislabeled POS Tags

NLTK				**CoreNLP**			
POS Tag	Incorrect	Total	Accuracy	POS Tag	Incorrect	Total	Accuracy
VB	1694	2243	24.48%	VB	1269	2243	43.42%
NNP	815	816	0.12%	NNP	754	816	7.60%
NN	776	6165	87.41%	NN	542	6165	91.21%
JJ	515	1235	58.30%	JJ	411	1235	66.72%
FW	229	229	0.00%	FW	225	229	1.75%
OpenNLP				**spaCy**			
POS Tag	Incorrect	Total	Accuracy	POS Tag	Incorrect	Total	Accuracy
VB	1384	2243	38.30%	NN	1648	6165	73.27%
NNP	811	816	0.61%	VB	1144	2243	49.00%
NN	612	6165	90.07%	JJ	501	1235	59.43%
JJ	473	1235	61.70%	NNP	458	816	43.87%
FW	226	229	1.31%	FW	229	229	0.00%
Flair				**Stanza**			
POS Tag	Incorrect	Total	Accuracy	POS Tag	Incorrect	Total	Accuracy
VB	1191	2243	46.90%	VB	918	2243	59.07%
NNP	665	816	18.50%	NNP	729	816	10.66%
NN	596	6165	90.33%	JJ	410	1235	66.80%
JJ	407	1235	67.04%	NN	395	6165	93.59%
FW	222	229	3.06%	FW	228	229	0.44%

Table 7 only presents the top 5 most frequently mislabeled POS tags. From this table, we make the following observations:

— Verb (VB), proper noun (NNP), noun (NN), adjective (JJ), and project-related abbreviation (FW) are the most frequently mislabeled tags. For each of the six evaluated taggers, they are the top 5 most frequently mislabeled tags. One possible reason is that they are much more popular in identifiers than other tags. In all identifiers involved in the evaluation, verb, proper

noun, noun, adjective, and project-related abbreviations account for 17.80%, 6.47%, 48.92%, 9.80%, and 1.82% of the tokens in identifiers, respectively. The evaluation results also suggest that researchers should pay special attention to such tags in the future to improve the overall performance of the POS taggers.

- The performance of the six taggers varies substantially on the most frequently mislabeled POS tags. For example, the accuracy on NNP varies from 0.12% (NLTK) to 43.87% (spaCy), although their overall performance on the whole dataset is comparable (as shown in Tables 2 and 3). It may suggest that these taggers could be complementary to each other in tagging tokens with different POS tags.
- The performance on the most frequently mislabeled POS tags is not necessarily poor. For example, the accuracy of CoreNLP, OpenNLP, Flair, and Stanza on NN is higher than 90%, although NN is the third most frequently mislabeled tag. Such tags are ranked at the top (according to the frequency of mislabeling) because of their high popularity. Notably, we do not list the top 5 hardest-to-label POS tags (in terms of accuracy) because it would result in rare tags like JJR (accounting for 0.01% of the tokens). Improving the performance on such rare tags does not significantly improve the overall performance of the taggers. In contrast, focusing more on the most frequently mislabeled tags has a greater chance of improving the overall performance of the taggers.

Besides the fine-grained token-level analysis, we also investigated how the POS patterns influence the performance of the evaluated POS taggers. The results are presented in Table 8. Notably, a POS pattern should cover all tokens in a single identifier. For example, if the name of a method is composed of a verb (VB) and a noun (NN), we say that the name follows the pattern "VB NN." If it contains any other tokens, it does not follow the pattern. From Table 8, we observe the following findings:

- The most frequently mislabeled patterns are "VB NN," "VB NN NN," and "NNP" (appear in all six taggers). We also notice that such POS patterns are popular in identifiers, accounting for 8.52%, 6.15%, and 2.71% of the identifiers, respectively. The patterns ranked on the top in Table 8 are mislabeled more frequently.
- By comparing Table 8 against Table 3, we observe that for NLTK, the identifier-level accuracy (*EM*) in terms of the patterns in Table 8 is substantially lower than the average accuracy of the tagger. For instance, the average identifier-level Exact Match of NLTK is 34.41%. However, its accuracy on patterns "VB NN," "VB NN NN," "NNP," "NNP NN," and "VB" is 23.67%, 24.75%, 0.00%, 0.00%, and 11.61%, respectively. This disparity suggests that NLTK's performance is not only influenced by the frequency of these patterns but also highlights a potential weakness in accurately tagging frequent identifier patterns. The results indicate that NLTK may require targeted improvements for handling these common patterns, as they significantly impact the tagger's overall effectiveness.

All such findings may suggest that improving the accuracy of such patterns in the future can significantly improve the overall performance of the taggers because such patterns are common and there is huge room for improvement on such patterns.

Finding 3: The performance of the POS taggers deserves substantial improvement on some popular tags like verbs, proper nouns, nouns, adjectives, and project-related abbreviations. It is critical in the future to improve the tagging performance on some popular but hard-to-tag POS patterns like "VB NN," "VB NN NN," and "NNP."

Table 8. Top 5 Most Frequently Mislabeled POS Patterns

NLTK				**CoreNLP**			
POS Pattern	Incorrect	Total	Accuracy	POS Pattern	Incorrect	Total	Accuracy
VB NN	316	414	23.67%	VB NN	217	414	47.58%
VB NN NN	225	299	24.75%	VB NN NN	166	299	44.48%
VB	137	155	11.61%	NNP NN	124	126	1.59%
NNP	132	132	0.00%	NNP	107	132	18.94%
NNP NN	126	126	0.00%	VB	77	155	50.32%
OpenNLP				**spaCy**			
POS Pattern	Incorrect	Total	Accuracy	POS Pattern	Incorrect	Total	Accuracy
VB NN	245	414	40.82%	VB NN	221	414	46.62%
VB NN NN	164	299	45.15%	VB NN NN	181	299	39.46%
NNP	132	132	0.00%	NN NN	162	403	59.80%
NNP NN	126	126	0.00%	NN	146	372	60.75%
VB	109	155	29.68%	NNP	92	132	30.30%
Flair				**Stanza**			
POS Pattern	Incorrect	Total	Accuracy	POS Pattern	Incorrect	Total	Accuracy
VB NN	205	414	50.48%	VB NN	128	414	69.08%
VB NN NN	163	299	45.48%	NNP NN	119	126	5.56%
NNP NN	95	126	24.60%	NNP	102	132	22.73%
NNP	94	132	28.79%	VB NN NN	92	299	69.23%
NN	86	372	76.88%	JJ NN	67	198	66.16%

4.4 RQ4: Reasons for Performance Reduction

To explore the reasons why POS taggers designed for and trained with natural language sentences are less accurate on identifiers, we analyze the difference between identifiers and natural language sentences. The difference is explained in the following subsections concerning their length, syntactic structures, POS distribution, and special tags.

4.4.1 Length of Items. Figure 1 presents the length distribution of three types of identifiers and natural language sentences. On the left side of the figure, we compare the length distribution of identifiers against that of natural language sentences. To intuitively display the different distributions of identifier types, we also enlarge the figure on the right side to show the length distributions of method names, parameter names, class names, and all identifiers combined.

From Figure 1, we make the following observations:

- Natural language sentences are much longer than identifiers. On one side, the average length of identifiers is 2.59 tokens, whereas natural language sentences on average have 16.36 tokens. The latter is $6.32 \approx 16.36/2.59$ times than the former. On the other side, the maximal length of identifiers is 9 tokens, which is even smaller than the median (14 tokens) of sentences' length.
- Concerning the length distribution, method names are similar to class names, whereas parameter names are substantially different from them. The plot on the right side of Figure 1 visualizes the differences (and similarities) among them. From the figure, the difference between parameter names and others (method names and class names) is obvious: Parameter names have a large/thick bottom and a long but thin tail. The reason is that the majority of parameter names consist of a singular token. In contrast, method names and class names typically comprise more tokens.

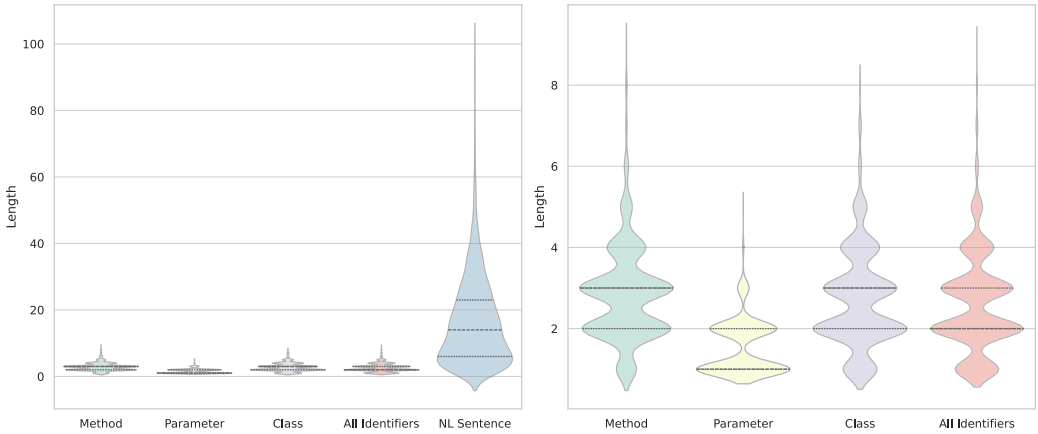

Fig. 1. Distribution of items' length.

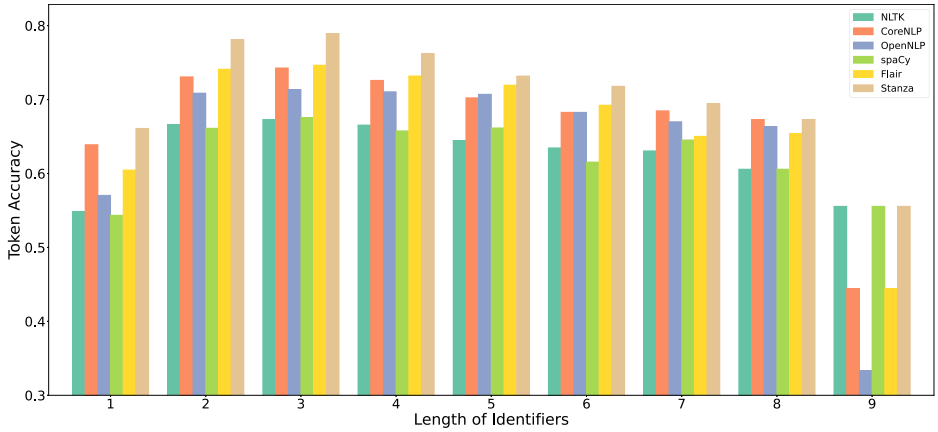

Fig. 2. Impact of identifiers' length on taggers' performance.

Figure 2 illustrates how identifiers' length influences the performance of the evaluated POS taggers. From this figure, we observe that the token-level accuracy Acc_{Token} first increases with the increase of identifiers' length and then decreases with the further increase in identifiers' length. That is, the taggers are less accurate in tagging identifiers that are either extremely short or extremely long. The possible reason for the low performance on the shortest items (whose length equals one) is that such items are often composed of a single token, whereas the taggers often employ sequence labeling models to assign tags to tokens. These models utilize the information of the nearby tokens or the whole sequences as factors to predict the tag of the specified token. However, single-token identifiers (consisting of a single token) lack substantial sequence information, making them less suitable for these models.

4.4.2 POS Distribution. Source code identifiers have their own unique POS distribution that is different from that of natural language sentences. The difference is also a potential reason for the low performance of natural language-specific POS taggers on source code identifiers. To visualize the difference, we illustrated the POS distribution for both identifiers and natural language sentences through Figures 3 and 4. From the figures, we make the following observations:

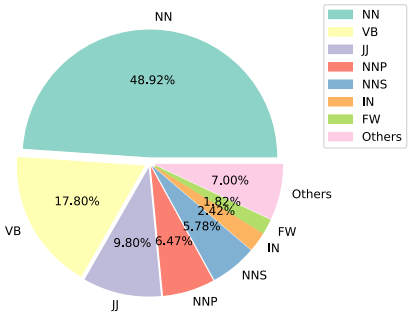

Fig. 3. POS distribution of identifiers.

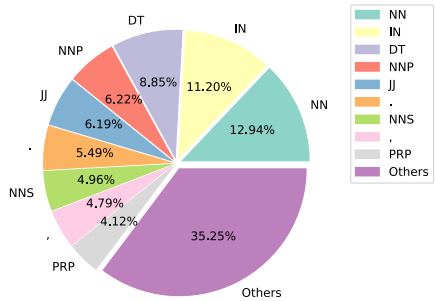

Fig. 4. POS distribution of natural language sentences.

- The POS distribution within identifiers is markedly uneven, with only seven categories surpassing 1.5%, whereas the other 16 POS tags account for less than 1.5%. The most prevalent tag ("noun," NN) alone accounts for around half (48.92%) of the tokens—a proportion significantly higher than what is typically seen in natural language sentences (i.e., 12.94%). The top 5 most frequent tags together account for 88.77% of the tokens, indicating that taggers focusing on these predominant tags might yield promising results.
- The distribution of tags in natural language sentences exhibits a more even spread. Although NN is also the most common tag in natural language sentences, it accounts for only 12.94% of sentence tokens. The top 5 most frequent tags in natural language sentences in the total amount account for 45.40% of the tokens, also substantially smaller than that (88.77%) in identifiers.
- Unlike natural language sentences, identifiers contain numerous special tokens that are challenging to tag. Our analysis reveals that 1.82% of identifier tokens are project-specific abbreviations (FW). These abbreviations frequently appear in the most commonly mislabeled POS and patterns, indicating that current POS taggers are often unable to tag such abbreviations accurately.

4.4.3 Syntactic Structure. To investigate the differences in syntactic structures between identifiers and natural language sentences, we compiled the top 5 patterns from each type and analyzed their structural complexity through constituency trees for a more intuitive comparison. Additionally, we summarized previous studies on identifier structures and validated their applicability on IDData.

Findings in Our Datasets. We retrieve the top 5 most popular POS patterns from identifiers and natural language sentences, respectively. Our evaluation results suggest that:

- The structural patterns in identifiers are much more concentrated than those in natural language sentences. For the labeled identifiers, the total number of POS patterns is 786. In contrast, for the 3,000 natural language sentences, there are 2,649 patterns.
- The top 5 most popular POS patterns in identifiers are "VB NN" (covering 8.52% of the identifiers), "NN NN" (8.29%), "NN" (7.65%), "VB NN NN" (6.15%), and "JJ NN" (4.07%). In contrast, the top 5 most popular POS patterns in natural language sentences are "UH" (3.53%), "JJ" (0.53%), "NN" (0.5%), "NNP, NNP CD, CD" (0.43%), and "NNS" (0.43%). We notice that there is no overlap at all between the two sets of most popular patterns, which may suggest that identifiers and natural language sentences do have significant differences concerning their syntactic structures.
- The top 5 most popular POS patterns in identifiers cover 34.68%, whereas the number is substantially reduced to 5.43% on natural language sentences. The former is six times higher

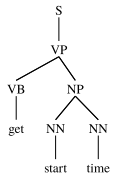

Fig. 5. A constituency tree for method names.

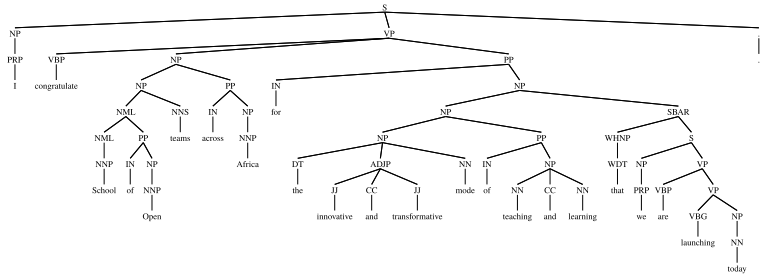

Fig. 6. A constituency tree for natural language sentences.

than the latter, which further confirms the finding that identifiers are more regular concerning POS patterns.

To provide a more intuitive comparison of the structures in identifiers and natural language sentences, and to better understand the complexity of these structures, we use the Berkeley Neural Parser [30] to generate constituency trees for both identifiers and natural language sentences. This approach allows us to extract structural information from the parsed data. To simplify the explanation, we present two example parsing trees in Figure 5 (for a method name "getStartName") and Figure 6 (for a natural language sentence "I congratulate School of Open teams across Africa for the innovative and transformative mode of teaching and learning that we are launching today.").

Our evaluation results suggest that the average depth of the consistency tree for identifiers is 2.91, whereas natural language sentences exhibit a significantly larger average depth of 7.53. This disparity underscores the greater complexity in the trees of natural language sentences compared to those of identifiers.

Relating IDData to Prior Studies on Identifier Structures. The gap between the structures of identifiers and natural language sentences is one of the key challenges in designing identifier-specific taggers. Previous studies have explored the unique structures of identifiers from two perspectives: the pattern level and the phrase level. The pattern level focuses on the order and combination of POS within identifiers, while the phrase level examines how tokens recursively form phrases.

At the *pattern level*, Newman et al. [46] identified the top 5 grammar patterns from their dataset. However, their use of a smaller custom tagset prevents direct validation on IDData. To assess the relevance of these patterns to our dataset, we mapped them to the Penn Treebank tagset and analyzed their frequency distribution within IDData. This allowed us to determine whether these patterns remain prevalent in our dataset. The identified patterns are as follows:

- Pattern 1: Identifiers that start with a noun or adjective and end with a noun. The popularity of this pattern is supported by the fact that both "NN NN" and "JJ NN" appear in the top 5 patterns of IDData, verifying its prevalence.
- Pattern 2: Identifiers that start with a verb, end with a noun, and include other nouns or adjectives in the middle. The occurrence of the pattern "VB NN NN" in the top 5 patterns of IDData confirms the popularity of this pattern.
- Pattern 3: Similar to Pattern 2 but ends with a plural noun. The pattern "VB NN NNS" at a frequency of 1.38%, while "VB JJ NNS" occurs at 0.93% in IDData. However, neither of these patterns appears in the top 5, suggesting that this pattern is less frequent and not entirely consistent with IDData.
- Pattern 4: A verb followed by a noun. "VB NN" is the most frequent pattern in IDData, confirming this pattern.

Table 9. Phrase Structures in IDData

Phrase	Explanation	Example Identifier	Example Structure
NP	Noun phrase	aggregate query	(NP (JJ aggregate) (NN query))
PP	Prepositional phrase	on conflict	(PP (IN on) (NP (NN conflict)))
VP	Verb phrase	starts with	(VP (VBZ starts) (IN with))
ADJP	Adjective phrase	less than or equal	(ADJP (JJR less) (IN than) (CC or) (JJ equal))
ADVP	Adverb phrase	merge local to proto	(S (VP (VB merge) (ADVP (JJ local)) (PP (IN to) (NP (NN proto)))))
WHADVP	Wh-adverb phrase	where node	(SBAR (WHADVP (WRB where)) (NP (NN node)))

— Pattern 5: A single noun. The pattern "NN" is also in the top 5 patterns of IDData, verifying this pattern.

In summary, most of these patterns are confirmed, with Pattern 3 being the only exception. Its popularity is less consistent with the findings in IDData, as it appears with a lower frequency in IDData.

Other studies focus on *phrase-level* structures to analyze identifiers. The SWUM Tagger [23] utilized **Noun Phrase (NP)**, **Verb Phrase (VP)**, and **Prepositional Phrase (PP)** structures derived from the Penn Treebank syntactic tagset [37, 40], which was originally designed for constituency parsing tasks [29]. In this framework, syntactic tags correspond to the non-leaf nodes of a constituency parsing tree, while POS tags are associated with the leaf nodes. Additionally, SWUM proposed three more structures: **Verb Group (VG)**, **Equivalence (EQ)**, and **Conjunctive Phrase (CP)**. However, these additional structures are excluded from our evaluation for the following reasons:

— VG: It is a simplified version of the recursive VP and is already tagged as VP by standard parsers.
— EQ: This represents a higher-level relationship between NP and VG, but it does not directly consist of tokens and is therefore not a standalone phrase structure.
— CP: Although proposed by SWUM, CP was not implemented in their work [23].

Consequently, we focus our analysis on NPs, VPs, and PPs to assess whether these structures are adequate to capture the full diversity of phrase structures in identifiers within the IDData dataset. To achieve this, we used the Berkeley Neural Parser to parse all identifiers in IDData and examined the parsed results for any additional phrase structures beyond NP, VP, and PP. This allows us to assess whether these three phrase types can comprehensively describe the syntactic structures found in identifiers.

We identified additional phrase structures not covered by prior works, namely **Adjective Phrase (ADJP)**, **Adverb Phrase (ADVP)**, and **Wh-adverb Phrase (WHADVP)**, as detailed in Table 9. After filtering out NP, VP, and PP structures, we manually validated the remaining phrase types and confirmed the presence of ADJP, ADVP, and WHADVP in IDData. This finding reveals that the phrase structures identified by earlier works are incomplete for this dataset.

The POSSE tagger [21] contains a phrase-level grammar specifically designed for parsing identifiers in source code. This grammar consists of six construction rules: Noun Modifier Phrase, NP, Verb Modifier, VG, Past Participle Phrase, and PP. Like other approaches, the foundation of these lexical phrases is derived from natural language grammar principles [37].

To validate this grammar, we applied POSSE's chunking model to the identifiers in IDData. Our findings show that 159 identifiers, accounting for approximately 3.27% of the dataset, failed to be

Table 10. Removal of FW (Token Accuracy)

Taggers	MData		PData		CData		IDData	
	-FW	Δ	-FW	Δ	-FW	Δ	-FW	Δ
NLTK	68.40%	0.71%	74.62%	7.46%	64.95%	0.70%	66.91%	1.18%
CoreNLP	77.46%	0.75%	74.38%	7.03%	71.03%	0.86%	73.65%	1.35%
OpenNLP	75.06%	0.78%	72.74%	6.85%	68.64%	0.82%	71.31%	1.31%
spaCy	74.20%	0.86%	58.75%	5.19%	63.01%	0.72%	66.85%	1.28%
Flair	78.81%	0.68%	69.92%	6.67%	71.38%	0.82%	74.02%	1.33%
Stanza	84.57%	0.81%	75.56%	7.03%	73.74%	0.89%	77.87%	1.44%

Table 11. Removal of FW (Exact Match)

Taggers	MData		PData		CData		IDData	
	-FW	Δ	-FW	Δ	-FW	Δ	-FW	Δ
NLTK	29.08%	0.51%	65.42%	10.06%	34.04%	1.18%	36.15%	1.64%
CoreNLP	49.97%	0.87%	66.31%	10.05%	43.10%	1.49%	48.23%	2.16%
OpenNLP	44.57%	0.78%	64.35%	9.60%	38.15%	1.32%	43.49%	1.93%
spaCy	45.52%	0.80%	48.48%	7.46%	35.12%	1.22%	40.26%	1.82%
Flair	52.57%	0.92%	61.85%	9.37%	43.70%	1.51%	48.90%	2.19%
Stanza	65.33%	1.14%	67.02%	10.31%	46.81%	1.62%	55.54%	2.51%

parsed using this grammar. This relatively small failure rate suggests that POSSE's grammar is capable of successfully parsing most identifiers in the dataset. However, despite its specific design for source code identifiers, POSSE's accuracy was observed to be lower than that of other natural language oriented taggers, as demonstrated in Tables 4–6. These results indicate that while POSSE's grammar is broadly applicable to identifiers, it may lack the precision and robustness necessary to compete with more advanced and widely adopted tagging models, particularly those leveraging sophisticated machine learning techniques.

4.4.4 Special Tags. One key distinction between natural language sentences and code identifiers lies in the special tags. For example, the tag FW, representing "foreign words" in NLP, is instead used to denote project-related abbreviations in code, which may not be captured effectively by natural language-oriented POS taggers. Additionally, proper nouns tagged as NNP or NNPS in code are often specific to computer science and software development domains, which may not be well represented in the training corpora of natural language-oriented POS taggers. This section examines the influence of these domain-specific tags on overall tagging performance.

Influence of FW. To assess the impact of project-related abbreviations (tagged as FW) on the performance of natural language-oriented POS taggers, we conducted an evaluation by removing tokens labeled with FW and reanalyzing the results. By excluding these project-related abbreviations, we aimed to isolate the core tagging performance of the taggers without interference from project-related jargon. The resulting performance metrics, presented in Tables 10 and 11, highlight the extent to which the presence of FW tokens affects both Token Accuracy and Exact Match. From the tables, we make the following observations:

— The presence of the FW tag negatively impacts the performance of POS taggers. After filtering out the identifiers containing FW, all taggers exhibited improvements in both Token Accuracy

Table 12. Categories of Proper Nouns

Category	Explanation	Example
Software Development	Program Language Idiomatic Phrases of Software Development	python, sql, mojo... argc, argv, foo...
Standard	Information Field Standard File Format	utf8, url, uri... json, html, csv...
Software/Library	Software Name Library/Project Name	hadoop, latex, excel... pandas, keras, numpy...
System/Hardware	Operating System Hardware	unix, io, fs... cpu...
Operator	Mathematical Operator	fft, sqrt, softmax...
Other	Proper Noun in Other Field	rnn, gan, lecun...

and Exact Match scores across various identifier types. Specifically, in the IDData dataset, Token Accuracy increased across models, ranging from 1.18% (NLTK) to 1.44% (Stanza), while Exact Match improved from 1.64% (NLTK) to 2.51% (Stanza). Although these improvements exist, the results remain below the performance observed in the NLData dataset, highlighting the persistent challenge of accurately tagging identifiers.

— In contrast to the minimal decline in accuracy observed for method names and class names (less than 2%), parameter names show a more substantial drop in performance. Token Accuracy decreases by 5.19% (spaCy) to 7.46% (NLTK), while Exact Match declines by 7.46% (spaCy) to 10.31% (Stanza). This significant discrepancy can be attributed to two primary factors: (1) the short average length of parameter names in PData (1.54 tokens), with a considerable portion of single-token identifiers being tagged as FW, which accounts for 6.94% (46 out of 663) of PData; and (2) a total of 663 identifiers in PData, 102 of which are tagged as FW, resulting in a much higher proportion of FW identifiers (15.38%) compared to the overall IDData dataset (4.52%). These findings highlight the sensitivity of tagger performance to short identifiers, which are more prevalent in parameter names, thereby posing challenges for accurate tagging.

Exploration of NNP/NNPS. The other special tag is the proper nouns (NNP/NNPS). As shown in Figure 3, NNP represents 6.47% (816/12,603) of all tokens in the IDData, making it the fourth most frequent tag. However, NNP is also identified in Table 7 as one of the top 5 most frequently mislabeled POS tags. Unlike the NN (noun) tag, which appears in Table 7 due to its large proportion, the accuracy of tagging NNP varies significantly across models, ranging from as low as 0.12% with NLTK to 43.87% with spaCy. This indicates that natural language-oriented POS taggers struggle to identify proper nouns in IDData correctly. To better understand this issue, we filtered out 761 identifiers in IDData and conducted a manual analysis of 830 tokens (816 NNP + 14 NNPS). This analysis aims to identify categories of proper nouns, helping propose effective tagging methods for these identifiers.

Building on the definition of software-specific named entities proposed by Ye et al. [61], we define six categories of proper nouns identified in the IDData dataset: Software Development, Standard, Software/Library, System/Hardware, Operator, and Other. These categorized proper nouns provide a structured understanding of NNP/NNPS usage in IDData. A summary of these categories is presented in Table 12.

We further analyzed the distribution of the identified proper noun categories, as depicted in Figure 7. The resulting proportions highlight notable variability across categories: Other (25.03%), Operator (22.50%), Standard (18.65%), Software Development (14.56%), Software/Library (13.48%),

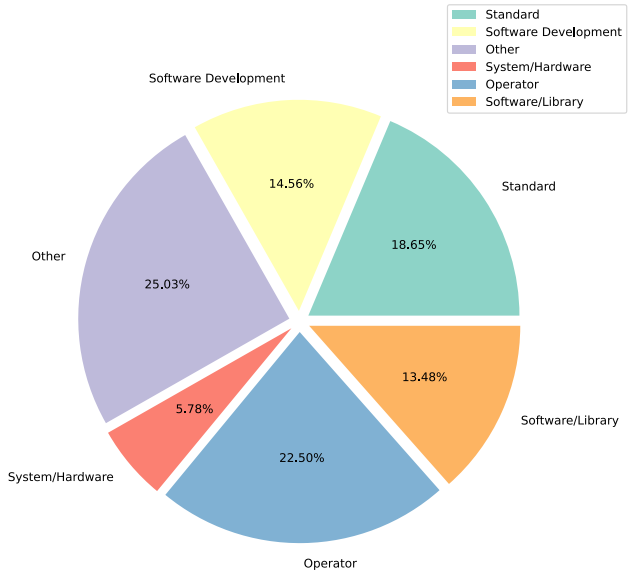

Fig. 7. Distribution of proper nouns.

and System/Hardware (5.78%). Except for the Other category, the Operator category occupies the largest proportion, primarily due to the frequent use of domain-specific mathematical operations. These operators, such as fft, sqrt, and softmax, are integral components in various computational libraries for machine learning and data analysis. The categories Standard, Software Development, and Software/Library show similar proportions, suggesting a balance between the need for file operations, the usage of development idioms, and the reliance on external software libraries. The System/Hardware category has the smallest proportion, suggesting that the selected projects are more focused on interactions with software components (e.g., files, libraries) rather than lower-level systems or hardware. This implies that most of the projects in IDData are designed for higher-level application development rather than system programming or hardware interfacing. The accurate tagging of these categories could be significantly enhanced through the use of field-specific dictionaries.

> *Finding 4:* Identifiers are substantially different from natural language sentences, concerning their length, POS distribution, syntactic structures, and special tags. Consequently, the natural language-oriented POS taggers specially designed for natural languages and trained with natural language sentences do not perform well on identifiers, especially single-token identifiers.

5 Improving Natural Language-Oriented POS Taggers with Identifier-Specific Data

As highlighted in Section 4.2, natural language-oriented POS taggers are often less accurate on source code identifiers. The major reason is that these taggers are designed for and trained on natural language sentences, whereas identifiers fundamentally differ from natural languages. Consequently, an intuitive and potentially fruitful way to improve the performance of POS taggers on code identifiers is to retrain the taggers with identifier POS datasets.

We only retrieve and label method names because of the following reasons. First, it is time and resource consuming to label a large number of identifiers. The annotation of MNTrain took

Table 13. Training and Validation Data (MNTrain)

Dataset	Project	Size	#Tokens
Training Set	Cassandra	547	1,812
	Elasticsearch	557	1,774
	Gradle	588	1,884
	Hibernate	666	2,430
	IntelliJ IDEA Community Edition	699	2,179
	Liferay Portal	595	2,563
	Presto	571	1,819
	Spring Framework	709	2,864
	WildFly Application Server	656	2,279
	TOTAL	*5,588*	*19,604*
Validation Set	libGDX	577	1,651
MNTrain		6,165	21,255

our three-annotator team 105 days. The cost of annotating all types of identifiers is prohibitively high. Second, method names are crucial for understanding the functions of the enclosing classes. Consequently, we only label a single type of identifier (method names).

In this section, we quantitatively investigate the feasibility of retraining by answering the following research questions:

— *RQ5*: To what extent can we improve the performance of the natural language-oriented POS taggers by training them with the identifier POS dataset?
— *RQ6*: Are POS taggers trained on method names effective for other types of identifiers?

5.1 Training with Method Name Corpus

To build the dataset for training, we retrieved method names from an existing dataset called Java-Small [3]. Java-Small is a well-known Java dataset consisting of 11 popular Java projects, i.e., Cassandra, Elasticsearch, Gradle, Hibernate ORM, IntelliJ IDEA Community Edition, Liferay Portal, Presto, Spring Framework, WildFly Application Server, libGDX, and Hadoop. As Hadoop has been employed for building the testing dataset in Section 3, we excluded it for training data construction. The details are listed in Table 13.

We construct the training and validation dataset (MNTrain) with the selected 10 projects in *Java-small* exactly in the same way by the same participants as we built the testing dataset in Section 3. We divide the data into a training set and a validation set. After splitting, we further refined the training data by removing duplicate items and excluding any items that appeared in IDData (the testing dataset) from MNTrain. This step ensures that the evaluation remains unbiased and that no overlap exists between the training and testing data, preserving the integrity of the results. The final configuration of the MNTrain dataset is detailed in Table 13.

We train each of the evaluated taggers with their officially recommended training API or command-line instructions. Notably, the Ensemble Tagger is excluded from the evaluation because it only provides a pre-trained serialized model for inference and lacks the code for training the model. The heuristic taggers, i.e., SWUM and POSSE, are excluded as well because they do not require training. The natural language tagger Stanza was also excluded from our study. Stanza is specifically designed to align with the UD corpus, which necessitates a comprehensive dataset

Table 14. Taggers Information

Taggers	Model	Technique
NLTK	Perceptron	Statistical learning
CoreNLP	Maxent	Statistical learning
OpenNLP	Maxent	Statistical learning
spaCy	HashEmbedCNN	Deep learning
Flair	LSTM-CRF	Deep learning

Table 15. Token Accuracy after Retraining

Taggers	MData-Before	MData-After	Improvement	NLData
NLTK	67.69%	90.15%	**22.46%**	88.68%
CoreNLP	76.70%	87.01%	10.31%	95.01%
OpenNLP	74.29%	88.99%	14.70%	91.38%
spaCy	73.34%	89.38%	16.04%	95.52%
Flair	**78.13%**	**91.36%**	13.23%	**95.52%**

The bold font indicates the best-performing tagger for each dataset.

comprising both syntactic and morphological information, including parsing trees as illustrated in Figure 5. Instead of creating a new UD-format corpus encompassing full syntactic and morphological details, our research focuses on POS tagging. As a result, the evaluation only retrains NLTK, CoreNLP, OpenNLP, spaCy, and Flair. We list the model information of the taggers. This information is summarized in Table 14. NLTK, CoreNLP, and OpenNLP use Perceptron and Maxent models for POS tagging, which are statistical learning approaches. In addition, spaCy and Flair employ different types of neural networks, which are deep learning-based methods.

5.2 RQ5: Improving Performance by Retraining with Method Names

After the retraining with the method name corpus MNTrain, we apply the resulting models to the method names in the testing dataset MData (as introduced in Section 3.2) and compare the results against the off-the-shelf models on the same testing dataset (i.e., MData). The evaluation results are presented in Tables 15 and 16. Note that the columns "MData-Before" and "MData-After" present the performance before the retraining (i.e., trained with natural language sentences) and after the retraining (i.e., trained with method names), respectively. Column "improvement" is the performance improvement caused by the retraining, i.e., the performance after retraining minus the performance before retraining. Column "NLData" presents the performance of the taggers (without retraining) on natural language sentences.

From Tables 15 and 16, we make the following observations:

- All natural language POS taggers trained with method name corpus could significantly improve their performance on method names. The improvement in Token Accuracy varies from 13.23 percentage points (Flair) to 22.46 percentage points (NLTK), and the improvement in Exact Match varies from 20.34 percentage points (CoreNLP) to 46.85 percentage points (NLTK). All such results suggest that building a method name-specific POS dataset could substantially improve the performance of the state-of-the-art POS taggers, although such taggers are not specially designed for method names or other source code identifiers.

Table 16. Exact Match after Retraining

Taggers	MData-Before	MData-After	Improvement	NLData
NLTK	28.57%	75.42%	**46.85%**	29.60%
CoreNLP	49.10%	69.43%	20.34%	53.17%
OpenNLP	43.79%	73.42%	28.88%	33.17%
spaCy	44.73%	72.68%	33.56%	56.43%
Flair	**51.65%**	**78.29%**	26.64%	**58.37%**

The bold font indicates the best-performing tagger for each dataset.

- NLTK achieves the greatest improvement in both Token Accuracy and Exact Match. One possible reason is that NLTK has the lowest performance (concerning either Token Accuracy or Exact Match) before retraining, and thus, it has plenty of room for improvement. In contrast, Flair (the best tagger before retraining) has the smallest improvement in Token Accuracy.
- The performance on method names after retraining is comparable to that on natural language sentences. For example, the Token Accuracy of NLTK on method names is 90.15%, even higher than that (88.68%) on natural language sentences. Notably, however, for the most accurate approach on both two metrics, Flair, its Token Accuracy on method names (91.36%) is still slightly lower than that on natural language sentences (95.52%).

Finding 5: Retraining with method name corpus can substantially improve the performance of the POS taggers on method names. After retraining, their performance on method names is comparable to that on natural language sentences. This may suggest that natural language-oriented POS taggers specially designed for natural languages have the potential to work accurately on source code identifiers if we have an identifier corpus for training.

5.3 RQ6: Cross-Type Prediction

The evaluation results in Section 5.2 suggest that re-training the POS taggers with method name corpus can significantly improve their performance in tagging (predicting) method names. However, it remains unclear whether the taggers trained with method name corpus are accurate in predicting other types of identifiers, e.g., class names and parameter names. If yes, we do not have to create the training corpus for different types of identifiers, and the method name corpus alone could be used to train a POS tagger that works on all types of identifiers. To test the hypothesis, we apply the POS taggers (trained with method name corpus) in Section 5.2 to parameter names and class names in testing data PData and CData (as introduced in Section 3.2). The evaluation results are summarized in Tables 17 and 18. These tables present the Token Accuracy and Exact Match for the cross-type prediction task. By comparing the performance of the default taggers with the retrained taggers, we can observe improvements in the parameter names (PData) and class names (CData) after retraining.

From the results presented in Tables 17 and 18, we can draw the following insights:

- Some of the taggers benefit from utilizing method name training data (MNTrain) for cross-type prediction tasks. The majority of taggers demonstrate significant improvements in both Token Accuracy and Exact Match metrics for class names, with the exception of CoreNLP. The enhancement in Token Accuracy ranges from an increase of 0.03 percentage points (CoreNLP) to 10.88 percentage points (spaCy). Similarly, the Exact Match improvement spans

Table 17. Performance of Cross-Type Prediction (Token Accuracy)

Tagger	Default		Retrained		Improvement		NLData
	PData	CData	PData	CData	PData	CData	
NLTK	67.16%	64.25%	59.43%	73.66%	−7.72%	9.40%	88.68%
CoreNLP	**67.35%**	70.17%	58.65%	70.20%	−8.70%	0.03%	95.02%
OpenNLP	65.88%	67.82%	59.63%	72.57%	−6.26%	4.75%	91.38%
spaCy	53.57%	62.29%	63.93%	73.14%	10.36%	**10.88%**	**95.52%**
Flair	63.25%	**70.55%**	**74.88%**	**79.06%**	**11.63%**	8.51%	95.52%

The bold font indicates the best-performing tagger for each dataset.

Table 18. Performance of Cross-Type Prediction (Exact Match)

Tagger	Default		Retrained		Improvement		NLData
	PData	CData	PData	CData	PData	CData	
NLTK	55.35%	32.86%	41.93%	43.57%	−13.42%	10.71%	29.60%
CoreNLP	**56.26%**	41.60%	41.93%	37.67%	−14.33%	−3.93%	53.20%
OpenNLP	54.75%	36.83%	43.14%	42.64%	−11.61%	5.82%	37.43%
spaCy	41.03%	33.90%	47.36%	43.80%	6.33%	9.90%	56.47%
Flair	52.49%	**42.18%**	**64.10%**	**53.97%**	**11.61%**	**11.79%**	**58.37%**

The bold font indicates the best-performing tagger for each dataset.

from 5.82 percentage points (OpenNLP) to 11.79 percentage points (Flair). Additionally, spaCy and Flair exhibit substantial gains in performance on parameter names, with Token Accuracy improvements of 10.36 and 11.63 percentage points, respectively, and Exact Match enhancements of 6.33 and 11.61 percentage points.

- Three taggers (i.e., NLTK, CoreNLP, and OpenNLP) have decreased performance in both Token Accuracy and Exact Match on parameter names, compared against the default models (i.e., models trained with natural language sentences). This may suggest that these taggers still need type-specific data for training.
- There is still a performance gap between the cross-type prediction and natural language sentences. The cross-type prediction on identifiers remains substantially less accurate than the POS tagging on natural language sentences.

Finding 6: Training the natural language-oriented POS taggers with specific types of identifiers and applying it to other types of identifiers do not make the performance comparable to the performance on natural language sentences. It may suggest that we should construct a training dataset composed of various types of identifiers to make the POS taggers accurate on all kinds of identifiers.

6 Discussion

6.1 Task Formulation

As discussed in Section 2.1, POS tagging methods have evolved from rule-based to statistical learning-based, and finally to deep learning-based approaches. The overall trend in POS tagging has

increasingly moved away from relying on human-defined linguistic rules toward leveraging large-scale annotated datasets, driven by advancements in computational power and model architectures. Early identifier-specific taggers, such as SWUM and POSSE, employed rule-based methods that relied on a set of manually pre-defined rules. These rules were formulated from limited identifier data, and the grammar constructed from them followed the structure of context-free grammars [26]. In these approaches, the terminal symbols were restricted to entries in WordNet [21, 44]. However, the rigidity of these manually defined rules and restricted terminal sets limited the flexibility and adaptability of rule-based approaches.

Compared to learning-based, natural language-oriented taggers, the lower performance of the rule-based taggers, as demonstrated in Section 4.2, underscores their limitations. Additionally, the validation results in Section 4.4.3 further confirm that rule-based approaches fall short in adaptability and accuracy. In contrast, natural language-oriented taggers, particularly deep learning-based models, have achieved the state-of-the-art performance in POS tagging on both IDData and Newman's dataset. These models do not rely on pre-defined linguistic structures or external knowledge but instead learn the dependencies between token sequences implicitly from annotated data. As shown in the results, the trend points toward treating identifiers as a simple sequence of tokens rather than enforcing a rigidly structured grammar. An end-to-end deep learning model that maps token sequences directly to tag sequences is therefore the most effective approach.

6.2 Tagset Selection and Benchmark

Identifier-specific POS taggers often utilize simplified tagsets derived from the well-established Penn Treebank POS tagset (POS). Similarly, rule-based approaches such as SWUM and POSSE apply restricted phrase tagsets derived from the Penn Treebank syntactic tagset (Phrase). Since most identifiers in source code are composed of English words, and their readability stems from the natural language descriptions they represent, maintaining alignment with the Penn Treebank tagset offers distinct advantages. Using the same tagset as natural language corpora ensures that the taggers can leverage established linguistic structures, which improves the overall consistency and interpretability of results.

The evaluation results underscore these concerns. Both spaCy and Flair achieved impressive accuracy rates of 96.52% and 97.85%, respectively, on a dataset containing only 11 tags as its tagset. While these results are high, even outperform their performance on natural language datasets, indicating that may be less challenging and less informative to evaluate the capabilities of advanced taggers.

To address these concerns, we opted to adopt the full Penn Treebank POS tagset—the same one used in natural language corpora—and constructed a larger, more diverse test dataset, IDData. This decision eliminates the need for biased mappings and allows for a fairer evaluation of natural language-oriented taggers, providing a more appropriate and rigorous benchmark for POS tagging tasks in code-related contexts.

6.3 Sequences Only or Extra Knowledge?

POS tagging is fundamentally a sequence labeling task, where the goal is to map an input sequence of tokens to their corresponding POS tags [22]. The classical POS tagging framework requires only the input sequence to produce accurate tag outputs, and deep learning models have significantly advanced this capability. Modern deep learning approaches rely solely on input sequences without requiring explicit linguistic knowledge or additional metadata. As demonstrated in Sections 4.1 and 4.2, natural language-oriented POS taggers such as spaCy and Flair exhibit strong performance on both natural language data (NLData) and identifier datasets like Newman's dataset, confirming the robustness of these models even when minimal input is provided.

In contrast, identifier-specific POS taggers often require extra input features to enhance their performance. For example, identifier-specific taggers process more than just the token sequence of a method name; they rely on extra metadata such as the identifier type (e.g., "*FUNCTION*"), the returned value type (e.g., "*Histogram*"), and the full method signature (e.g., "*getDistribution(Histogram[][] block, JobOutcome outcome, LoggedJob.JobType type)*"). Different taggers also make different assumptions about the structure of identifiers, with some of them even considering source code structure in their determinations. As summarized in Section 4.4.3, Newman et al. identified the top 5 grammatical patterns, SWUM utilized six phrase structures, and POSSE applied six construction rules to incorporate source code structure into their analyses. Despite the inclusion of these additional features, natural language-oriented POS taggers, which only process the identifier token sequence (e.g., "*get distribution*"), significantly outperform identifier-specific taggers, both on IDData and Newman's dataset, as discussed in Section 4.2.

The superior performance of natural language-oriented POS taggers indicates that, for POS tagging tasks, extra information beyond the token sequence itself may have limited utility. Therefore, our assumption is to tag identifiers as plain token sequences without incorporating any additional structure information. However, this does not entirely negate the need for extra knowledge. As discussed in Section 4.4.4, certain project-related abbreviations or domain-specific proper nouns require specialized knowledge for correct tagging. Such tokens may not be easily handled by standard POS taggers without integrating extra resources, such as domain-specific dictionaries or knowledge bases. This concept is analogous to the way traditional NLP POS taggers benefit from combining linguistic knowledge (e.g., syntactic or morphological rules) with machine learning models [38]. In such cases, the integration of extra knowledge—whether through specialized lexicons or other contextual data—can still play a critical role in improving tagger performance for identifier POS tagging, particularly in technical or domain-specific contexts. We suggest that while deep learning models excel with minimal input, the strategic use of domain-specific knowledge remains relevant for improving performance in specialized cases.

6.4 Significance and Implications

The evaluation results indicate that current POS taggers still require substantial improvements. We summarize key implications derived from our findings below.

A Finer-Grained Benchmark Enhances the Exploration of Tagger Capabilities. Our newly introduced test dataset, IDData, provides a more granular evaluation framework with a broader tagset and extensive item coverage. By capturing subtle performance differences among taggers, IDData enables a deeper analysis of their strengths and limitations. The results demonstrate that this finer-grained benchmark offers valuable insights into identifier POS tagging, supporting more comprehensive assessments in future research.

Natural Language-Oriented Taggers with Domain-Specific Data Are Able for Identifier POS Tagging. As demonstrated in Section 4.2, natural language-oriented taggers perform better on both the mapped IDData and the original Newman's dataset. Notably, these taggers perform exceptionally well on Newman's dataset, even exceeding their performance on natural language sentences. With the same training data, natural language-oriented taggers outperform identifier-specific taggers significantly. Additionally, their performance on MNData approaches that on NLData when trained with the MNTrain dataset. These results suggest that natural language-oriented taggers are indeed effective for identifier POS tagging with domain-specific data.

Labeled Sequences Are All You Need: No Need for Extra Considerations. We analyze structural considerations in detail in Sections 4.4.3 and 6.3. Natural language-oriented taggers that treat identifiers as plain sequences consistently outperform identifier-specific taggers that incorporate structure and require additional information. Deep learning-based taggers demonstrate an ability

to capture the underlying knowledge needed to correctly assign tags without extra structural information. These results suggest that increasing labeled, domain-specific data is more beneficial for taggers than adding structural considerations.

Special Tags Are the Key Point for Further Improvement. Identifiers differ significantly from natural language sentences, with parameters being notably distinct from other identifier types. When designing taggers, special attention should be given to single-token identifiers. It may be more effective to address special tags like FW and NNP/NNPS, while incorporating external resources such as domain-specific dictionaries. This approach can improve accuracy in tagging specialized identifiers.

6.5 Threats to Validity

The first threat to external validity is the size of the labeled data. In our evaluation, we manually created a new identifier corpus (IDData) with manually labeled POS tags for each token in the corpus. The corpus is composed of 4,862 unique identifiers with three types in three languages. The training and validation dataset (MNTrain) contains 6,165 unique method names from 10 Java projects. It is challenging to substantially enlarge the dataset because creating the dataset involves extensive human participation. To improve its quality, especially its diversity, we collect identifiers from different projects in different domains and developed by different developers. Identifiers in Java, C, and Python were selected as testing data because they are the most popular programming languages.

The second threat to external validity is that we only evaluated the most popular natural language-oriented POS taggers, i.e., NLTK, CoreNLP, OpenNLP, spaCy, Flair, and Stanza. The conclusions drawn on such taggers may not hold for other taggers. To reduce the threats, we selected the six most well-known and the most widely used POS taggers. To the best of our knowledge, they represent the state of the art in this area. We also notice that the conclusions drawn in this article hold for all of the six taggers, suggesting that the conclusions have good generality.

The third threat to external validity is that the data for retraining only involve method names in 10 open source projects. While focusing exclusively on method names allows us to manage resources effectively and maintain a feasible workload for our annotators, it may introduce a bias in the results. This narrow scope might not fully generalize to all types of identifiers. Consequently, we design the RQ6 to validate taggers' cross-type prediction. We did not include other programming languages because we do not know if the limited labeled data (i.e., the training data) are enough for training a cross-programming language POS tagger. Such a cross-language model should request a much larger and diversified training dataset that is hard to build manually. If the retrained models are not accurate, it may not suggest that the retraining is not effective. The limited training data could be the actual reason.

The fourth threat to external validity is the exclusion of prior identifier POS tagging data from our training dataset, MNTrain. Specifically, we annotated sampled identifiers from projects in the Java-small dataset. Reusing Newman's dataset would require complete re-annotation to align with our approach. To address this, we decided to annotate the Java-small dataset, ensuring that MNTrain is comprehensive and better suited for downstream tasks, such as method name generation.

The first threat to construct validity is the potential inaccuracy of manual annotation, which could affect the evaluation. To mitigate this, we engaged multiple participants in the annotation process, achieving a Kappa coefficient of 0.967, indicating a high level of inter-annotator agreement. The participants were graduate students in computer science with expertise in NLP. Conflicting annotations were resolved through multi-person consensus, and additional validation steps were implemented to reduce errors. Furthermore, creating a larger dataset helps minimize the overall impact of any residual inaccuracies in annotation.

The second threat to construct validity is the interpretations of the mapping process in Section 4.2. Although mapping has been widely used in previous evaluations [46–48], the specific settings we used, proposed by Ensemble Tagger, may still have introduced some limitations from different interpretations, potentially affecting tagger performance. To mitigate this, we also evaluated using Newman's dataset, where all learning-based taggers were trained on the same dataset, eliminating the need for mapping. This approach supports the findings from the mapped IDData while reducing potential biases introduced by the mapping process.

7 Conclusion and Future Work

POS tags are natural and critical properties of tokens in both natural languages and source code. Although there are quite a lot of well-known POS taggers, most of such taggers are specially designed for and trained with natural languages. Consequently, they could be less accurate in source code than in natural languages. In this article, we first construct a large-scale dataset IDData, consisting 4,862 identifiers where each token is manually labeled with a POS tag. Based on the dataset, we conduct the first large-scale empirical study to investigate how well natural language-oriented state-of-the-art POS taggers work on identifiers, why they are less accurate, and how to further improve them.

Our evaluation results suggest that off-the-shelf POS taggers are less accurate on identifiers in source code compared to their reported performance on natural languages. The state-of-the-art POS taggers specially designed for source code identifiers are not more accurate than such natural language-oriented taggers. Then, we explore the performance reduction when POS taggers trained with natural languages are applied to identifiers and find that identifiers are substantially different from natural language sentences at length, POS distribution, syntactic structures, and special tags aspects. To improve the accuracy in tagging method names, we manually construct a large-scale training dataset MNTrain (containing 6,165 method names) and retain the natural language-oriented taggers. Their resulting performance has been improved substantially and is comparable to their corresponding performance in natural languages. We also evaluate the retrained taggers on other types of identifiers such as parameter names and class names. Some taggers achieve better performance after retraining, but their performance is still not comparable to the performance on natural language sentences. This article not only improves the state-of-the-art POS tagging of identifiers but also provides the first large-scale identifier corpus with manually annotated POS tags. This publicly available dataset can facilitate future research in this line.

We also present several key implications for researchers: (1) We construct a new benchmark with a larger scale and finer-grained labels, providing a foundation for more comprehensive evaluations of taggers in subsequent studies. (2) Domain-related training data have been demonstrated as an effective way to enhance the identifier POS tagging performance of natural language-oriented taggers. (3) Deep learning-based taggers can extract latent knowledge from plain-labeled sequences, eliminating the need for additional structural considerations. (4) Future research should prioritize improvements related to special tags.

In the future, we would like to extend the empirical study to source code identifiers in other languages, like Rust and Ruby. We will design a new tagger trained with our annotated datasets. Further enlarging the datasets created in this article could be fruitful as well.

Data Availability

The replication package, including all code and data, is publicly available in our GitHub repository [55]. The repository contains the following contents: Experimental Data, Source Code, and Experimental Results. Researchers can access and utilize these resources for replication, further analysis, and comparative studies in the field of identifier POS tagging and related tasks.

References

[1] Zahra Shakeri Hossein Abad, Oliver Karras, Parisa Ghazi, Martin Glinz, Guenther Ruhe, and Kurt Schneider. 2017. What works better? A study of classifying requirements. In *Proceedings of the 2017 IEEE 25th International Requirements Engineering Conference (RE)*. IEEE, 496–501.

[2] Alan Akbik, Tanja Bergmann, Duncan Blythe, Kashif Rasul, Stefan Schweter, and Roland Vollgraf. 2019. FLAIR: An easy-to-use framework for state-of-the-art Nlp. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (Demonstrations)*, 54–59.

[3] Miltiadis Allamanis, Hao Peng, and Charles Sutton. 2016. A convolutional attention network for extreme summarization of source code. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2091–2100.

[4] Reem S. Alsuhaibani, Christian D. Newman, Michael L. Collard, and Jonathan I. Maletic. 2015. Heuristic-based part-of-speech tagging of source code identifiers and comments. In *Proceedings of the 2015 IEEE 5th Workshop on Mining Unstructured Data (MUD)*. IEEE, 1–6.

[5] Evan L. Antworth. 1990. *PC-KIMMO: A Two-Level Processor for Morphological Analysis*. Summer Institute of Linguistics.

[6] Apache. 2024. Apache OpenNLP. Retrieved from https://opennlp.apache.org/

[7] Michele Banko and Robert C. Moore. 2004. Part-of-speech tagging in context. In *Proceedings of the 20th International Conference on Computational Linguistics (COLING '04)*, 556–561.

[8] Tanmay Bhowmik, Nan Niu, Juha Savolainen, and Anas Mahmoud. 2015. Leveraging topic modeling and part-of-speech tagging to support combinational creativity in requirements engineering. *Requirements Engineering* 20, 3 (2015), 253–280.

[9] Gérard Biau and Erwan Scornet. 2016. A random forest guided tour. *Test* 25, 2 (2016), 197–227.

[10] Dave Binkley, Matthew Hearn, and Dawn Lawrie. 2011. Improving identifier informativeness using part of speech information. In *Proceedings of the 8th Working Conference on Mining Software Repositories*, 203–206.

[11] Steven Bird. 2006. NLTK: The natural language toolkit. In *Proceedings of the COLING/ACL 2006 Interactive Presentation Sessions*, 69–72.

[12] Eric Brill. 1992. A simple rule-based part of speech tagger. In *Proceedings of a Workshop on Speech and Natural Language*, 112–116.

[13] Eric Brill. 1994. Some advances in transformation-based part of speech tagging. arXiv:cmp-lg/9406010.Retrieved from https://arxiv.org/abs/cmp-lg/9406010

[14] Eric Brill. 1995. Transformation-based error-driven learning and natural language processing: A case study in part-of-speech tagging. *Computational Linguistics* 21, 4 (1995), 543–565.

[15] Michael L. Collard, Michael John Decker, and Jonathan I. Maletic. 2013. srcML: An infrastructure for the exploration, analysis, and manipulation of source code: A tool demonstration. In *Proceedings of the 2013 IEEE International Conference on Software Maintenance*. IEEE, 516–519.

[16] Douglass Cutting, Julian Kupiec, Jan Pedersen, and Penelope Sibun. 1992. A practical part-of-speech tagger. In *Proceedings of the 3rd Conference on Applied Natural Language Processing*, 133–140.

[17] Marie-Catherine De Marneffe, Christopher D. Manning, Joakim Nivre, and Daniel Zeman. 2021. Universal dependencies. *Computational Linguistics* 47, 2 (2021), 255–308.

[18] Explosion. 2023. spaCy: Industrial-Strength NLP. Retrieved from https://spacy.io/

[19] The Python Software Foundation. 2019. ast—Abstract Syntax Trees. Retrieved from https://docs.python.org/3/library/ast.html

[20] Sharon Goldwater and Tom Griffiths. 2007. A fully Bayesian approach to unsupervised part-of-speech tagging. In *Proceedings of the 45th Annual Meeting of the Association of Computational Linguistics*, 744–751.

[21] Samir Gupta, Sana Malik, Lori Pollock, and K. Vijay-Shanker. 2013. Part-of-speech tagging of program identifiers for improved text-based software engineering tools. In *Proceedings of the 2013 21st International Conference on Program Comprehension (ICPC)*. IEEE, 3–12.

[22] Zhiyong He, Zanbo Wang, Wei Wei, Shanshan Feng, Xianling Mao, and Sheng Jiang. 2020. A survey on recent advances in sequence labeling from deep learning models. arXiv:2011.06727. Retrieved from https://arxiv.org/abs/2011.06727

[23] Emily Hill. 2010. *Integrating Natural Language and Program Structure Information to Improve Software Search and Exploration*. University of Delaware.

[24] Michael Hucka. 2018. Spiral: Splitters for identifiers in source code files. *Journal of Open Source Software* 3, 24 (Apr. 2018), 653. DOI: https://doi.org/10.21105/joss.00653

[25] JavaParser. 2019. JavaParser: Analyse, Transform and Generate Your Java Codebase. Retrieved from https://javaparser.org/

[26] Frederick Jelinek, John D. Lafferty, and Robert L. Mercer. 1992. *Basic Methods of Probabilistic Context Free Grammars*. Springer.

[27] Shuo Jin, Songqiang Chen, and Xiaoyuan Xie. 2021. Property-based test for part-of-speech tagging tool. In *Proceedings of the 2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 1306–1311.

[28] Jeesu Jung, Sangkeun Jung, Hyein Seo, Hyuk Namgung, and Sungryeol Kim. 2022. Sequence alignment ensemble with a single neural network for sequence labeling. *IEEE Access* 10 (2022), 73562–73570.
[29] Dan Jurafsky. 2000. *Speech and Language Processing*. Pearson Education India.
[30] Nikita Kitaev and Dan Klein. 2018. Constituency parsing with a self-attentive encoder. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2676–2686.
[31] Julian Kupiec. 1992. Robust part-of-speech tagging using a hidden Markov model. *Computer Speech and Language* 6, 3 (1992), 225–242.
[32] Guangjie Li, Hui Liu, and Ally S. Nyamawe. 2020. A survey on renamings of software entities. *ACM Computing Surveys* 53, 2 (2020), 1–38.
[33] Bill Yuchen Lin, Frank F. Xu, Zhiyi Luo, and Kenny Zhu. 2017. Multi-channel Bilstm-Crf model for emerging named entity recognition in social media. In *Proceedings of the 3rd Workshop on Noisy User-Generated Text*, 160–165.
[34] Wang Ling, Tiago Luís, Luís Marujo, Ramón Fernandez Astudillo, Silvio Amir, Chris Dyer, Alan W. Black, and Isabel Trancoso. 2015. Finding function in form: Compositional character models for open vocabulary word representation. arXiv:1508.02096. Retrieved from https://arxiv.org/abs/1508.02096
[35] Liyuan Liu, Jingbo Shang, Xiang Ren, Frank Xu, Huan Gui, Jian Peng, and Jiawei Han. 2018. Empower sequence labeling with task-aware neural language model. In *Proceedings of the 32rd AAAI Conference on Artificial Intelligence*, 5253–5260.
[36] Meili Lu, Xiaobing Sun, Shaowei Wang, David Lo, and Yucong Duan. 2015. Query expansion via wordnet for effective code search. In *Proceedings of the 2015 IEEE 22nd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*. IEEE, 545–549.
[37] Christopher D. Manning. 1999. *Foundations of Statistical Natural Language Processing*. The MIT Press.
[38] Christopher D. Manning. 2011. Part-of-speech tagging from 97% to 100%: Is it time for some linguistics? In *International Conference on Intelligent Text Processing and Computational Linguistics*. Springer, 171–189.
[39] Christopher D. Manning, Mihai Surdeanu, John Bauer, Jenny Rose Finkel, Steven Bethard, and David McClosky. 2014. The Stanford CoreNLP natural language processing toolkit. In *Proceedings of 52nd Annual Meeting of the Association for Computational Linguistics: System Demonstrations*, 55–60.
[40] Mitch Marcus, Beatrice Santorini, and Mary Ann Marcinkiewicz. 1993. Building a large annotated corpus of English: The Penn treebank. *Computational Linguistics* 19, 2 (1993), 313–330.
[41] Lluis Marquez, Lluis Padro, and Horacio Rodriguez. 2000. A machine learning approach to POS tagging. *Machine Learning* 39, 1 (2000), 59–91.
[42] Angel R. Martinez. 2012. Part-of-speech tagging. *Wiley Interdisciplinary Reviews: Computational Statistics* 4, 1 (2012), 107–113.
[43] James Mayfield, Paul McNamee, Christine Piatko, and Claudia Pearce. 2003. Lattice-based tagging using support vector machines. In *Proceedings of the 12th International Conference on Information and Knowledge Management*, 303–308.
[44] George A. Miller. 1995. WordNet: A lexical database for English. *Communications of the ACM* 38, 11 (1995), 39–41.
[45] Anthony J. Myles, Robert N. Feudale, Yang Liu, Nathaniel A. Woody, and Steven D. Brown. 2004. An introduction to decision tree modeling. *Journal of Chemometrics* 18, 6 (2004), 275–285.
[46] Christian D. Newman, Reem S. AlSuhaibani, Michael J. Decker, Anthony Peruma, Dishant Kaushik, Mohamed Wiem Mkaouer, and Emily Hill. 2020. On the generation, structure, and semantics of grammar patterns in source code identifiers. *Journal of Systems and Software* 170 (2020), 110740.
[47] Christian D. Newman, Michael J. Decker, Reem S. Alsuhaibani, Anthony Peruma, Mohamed Wiem Mkaouer, Satyajit Mohapatra, Tejal Vishnoi, Marcos Zampieri, Timothy J. Sheldon, and Emily Hill. 2021. An ensemble approach for annotating source code identifiers with part-of-speech tags. *IEEE Transactions on Software Engineering* 48, 9 (2021), 3506–3522.
[48] Wyatt Olney, Emily Hill, Chris Thurber, and Bezalem Lemma. 2016. Part of speech tagging Java method names. In *Proceedings of the 2016 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 483–487.
[49] Maja Popovic and Hermann Ney. 2006. POS-based word reorderings for statistical machine translation. In *Proceedings of the 5th International Conference on Language Resources and Evaluation*, 1278–1283.
[50] Peng Qi, Yuhao Zhang, Yuhui Zhang, Jason Bolton, and Christopher D. Manning. 2020. Stanza: A python natural language processing toolkit for many human languages. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*, 101–108.
[51] Adwait Ratnaparkhi. 1996. A maximum entropy model for part-of-speech tagging. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*, 133–142.
[52] Beatrice Santorini. 1990. Part-Of-Speech Tagging Guidelines for the Penn Treebank Project. (3rd Revision). *Technical Reports (CIS)*, 570.

[53] Yu Shi, Xi Zhang, and Ning Yu. 2023. Pl-transformer: A POS-aware and layer ensemble transformer for text classification. *Neural Computing and Applications* 35, 2 (2023), 1971–1982.
[54] Catherine Soanes and Angus Stevenson. 2004. *Concise Oxford English Dictionary*. Vol. 11. Oxford University Press, Oxford.
[55] Hanlin Tang. 2025. TaggerEva. Retrieved from https://github.com/hanlintang/TaggerEva
[56] Yuan Tian and David Lo. 2015. A comparative study on the effectiveness of part-of-speech tagging techniques on bug reports. In *Proceedings of the 2015 IEEE 22nd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*. IEEE, 570–574.
[57] Yanxiang Tong, Yu Zhou, Lisheng Fang, and Taolue Chen. 2015. Towards a novel approach for defect localization based on part-of-speech and invocation. In *Proceedings of the 7th Asia-Pacific Symposium on Internetware*, 52–61.
[58] Kristina Toutanova, Dan Klein, Christopher D. Manning, and Yoram Singer. 2003. Feature-rich part-of-speech tagging with a cyclic dependency network. In *Proceedings of the 2003 Human Language Technology Conference of the North American Chapter of the Association for Computational Linguistics*, 252–259.
[59] Kristina Toutanvoa and Christopher D. Manning. 2000. Enriching the knowledge sources used in a maximum entropy part-of-speech tagger. In *Proceedings of the 2000 Joint SIGDAT Conference on Empirical Methods in Natural Language Processing and Very Large Corpora*, 63–70.
[60] Christoph Treude and Martin P. Robillard. 2016. Augmenting API documentation with insights from stack overflow. In *Proceedings of the 38th International Conference on Software Engineering*, 392–403.
[61] Deheng Ye, Zhenchang Xing, Chee Yong Foo, Zi Qun Ang, Jing Li, and Nachiket Kapre. 2016. Software-specific named entity recognition in software engineering social content. In *Proceedings of the 2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*. IEEE, 90–101.
[62] Amir Zeldes. 2017. The GUM corpus: Creating multilayer resources in the classroom. *Language Resources and Evaluation* 51, 3 (2017), 581–612. DOI: https://doi.org/10.1007/s10579-016-9343-x
[63] Yutong Zhao, Lu Xiao, Pouria Babvey, Lei Sun, Sunny Wong, Angel A. Martinez, and Xiao Wang. 2020. Automatically identifying performance issue reports with heuristic linguistic patterns. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 964–975.
[64] Yu Zhou, Yanxiang Tong, Taolue Chen, and Jin Han. 2017. Augmenting bug localization with part-of-speech and invocation. *International Journal of Software Engineering and Knowledge Engineering* 27, 6 (2017), 925–949.
[65] Yu Zhou, Xin Yan, Wenhua Yang, Taolue Chen, and Zhiqiu Huang. 2019. Augmenting Java method comments generation with context information based on neural networks. *Journal of Systems and Software* 156 (2019), 328–340.

Received 14 June 2024; revised 3 June 2025; accepted 4 June 2025